

A unified heuristic and an annotated bibliography for a large class of earliness–tardiness scheduling problems

Arthur Kramer^{1,2}  · Anand Subramanian³

Published online: 15 December 2017
© Springer Science+Business Media, LLC 2017

Abstract This work proposes a unified heuristic algorithm for a large class of earliness–tardiness (E–T) scheduling problems. We consider single/parallel machine E–T problems that may or may not consider some additional features such as idle time, setup times and release dates. In addition, we also consider those problems whose objective is to minimize either the total (average) weighted completion time or the total (average) weighted flow time, which arise as particular cases when the due dates of all jobs are either set to zero or to their associated release dates, respectively. The developed local search-based metaheuristic framework is quite simple, but at the same time relies on sophisticated procedures for efficiently performing local search according to the characteristics of the problem. We present efficient move evaluation approaches for some parallel machine problems that generalize the existing ones for single machine problems. The algorithm was tested in thousands of instances of sev-

eral E–T problems and particular cases. The results obtained show that our unified heuristic is capable of producing high-quality solutions when compared to the best ones available in the literature that were obtained by specific methods. Moreover, we provide an extensive annotated bibliography on the problems related to those considered in this work, where we not only indicate the approach(es) used in each publication, but we also point out the characteristics of the problem(s) considered. Beyond that, we classify the existing methods in different categories so as to have a better idea of the popularity of each type of solution procedure.

Keywords Earliness–tardiness · Heuristic framework · Iterated Local Search · Survey

1 Introduction

Scheduling problems have been widely studied in the literature over the past 50 years, receiving considerable attention from many scholars and practitioners around the world. Motivated by complex real-life problems faced by different types of companies, as well as by the challenge in solving them, a huge number of scheduling variants and solution procedures were proposed during this half-century period. Hence, it is safe to say that *Scheduling* is one of the most important subjects in the fields of Operations Research and Management Science.

A particular branch of the scheduling theory arises in the context of Just-In-Time (JIT) manufacturing systems. Problems of this branch of scheduling are generally referred to as earliness–tardiness (E–T) scheduling problems, where penalties are incurred if a job is completed before or after its due date. Moreover, they often include several additional features commonly found in the scheduling literature such as multi-

Electronic supplementary material The online version of this article (<https://doi.org/10.1007/s10951-017-0549-6>) contains supplementary material, which is available to authorized users.

✉ Arthur Kramer
arthurhfrk@gmail.com; arthur.kramer@unimore.it
Anand Subramanian
anand@ci.ufpb.br

- ¹ Departamento de Engenharia de Produção, Centro de Tecnologia, Universidade Federal da Paraíba, Campus I - Bloco G, Cidade Universitária, João Pessoa, PB 58051-970, Brazil
- ² Dipartimento di Scienze e Metodi dell'Ingegneria, Università degli Studi di Modena e Reggio Emilia, Via Amendola 2, Pad. Morselli, Reggio Emilia 42122, Italy
- ³ Departamento de Sistemas de Computação, Centro de Informática, Universidade Federal da Paraíba, Rua dos Escoteiros, s/n, Mangabeira, João Pessoa, PB 58058-600, Brazil

ple machines (identical, uniform, unrelated), release dates, sequence-dependent setup times, among others.

Weighted E–T problems are usually $\mathcal{NP}$ -hard, since they include the single machine total weighted tardiness scheduling problem ($1||\sum w_j T_j$) as a particular case, which is known to be $\mathcal{NP}$ -hard (Lawler 1977; Lenstra et al. 1977). Therefore, solving this type of problems to optimality is an extremely hard task. Nevertheless, there has been a continuous effort toward the development of efficient exact algorithms for this type of problems (Sourd and Kedad-Sidhoum 2003, 2008; Sourd 2005, 2009; Tanaka and Fujikuma 2008, 2012; Tanaka et al. 2009; Pessoa et al. 2010; Tanaka 2012). In some particular situations, often involving a single machine environment without sequence-dependent setup times, they seem to be very effective, even solving instances with up to 300 jobs (Tanaka et al. 2009; Tanaka 2012), but in most cases their application is still limited to small/medium-sized instances. Heuristic algorithms are thus the natural alternative for trying to generate high-quality feasible solutions within an acceptable computational time.

The large majority of heuristic algorithms proposed for scheduling problems were devised for a particular variant. While there are unified methods for polynomial problems (Leyvand et al. 2010) as well as polynomial time approximation schemes (Kumar et al. 2009; Epstein et al. 2013) and exact approaches (Chen and Powell 1999; Tanaka and Fujikuma 2008, 2012; Lin 2013) for classes of $\mathcal{NP}$ -hard problems, we are not aware of general heuristics developed for similar purposes. Furthermore, we strongly believe that unified methods are extremely important in practice. For example, commercial packages must be designed to be both robust and general enough to efficiently deal with many real-life problems. Also, these type of methods can be a good source of reference if one intends to evaluate the performance of specially tailored algorithms for some particular variants.

This work attempts to start filling the aforementioned gap by proposing a unified heuristic algorithm for single and parallel machine scheduling problems. By “unified” we mean that several specific ingredients are put together into a single framework that is capable of efficiently solving a large class of distinct scheduling problems. However, due to the countless number of variants existing in the literature, we decided to focus our attention on a subset of them, in particular, (E–)T problems without preemption. We consider a class of problems that may or may not consider earliness penalties, sequence-dependent setup times, release dates, due dates and idle time, resulting in a broad range of variants that include other type of objectives such as minimizing the total weighted tardiness; total (average) weighted completion time; and total (average) weighted flow time.

The developed heuristic algorithm extends the one of Subramanian (2012) and Subramanian et al. (2013) that was successfully applied to solve a class of vehicle routing problems.

The conceptual idea of the proposed method is quite simple. However, in contrast to the original algorithm, some specific parts rely on sophisticated procedures (Ibaraki et al. 2005, 2008; Ergun and Orlin 2006; Liao et al. 2012) for efficiently performing local search according to the characteristics of the problem. One particular contribution of this work is the development of efficient move evaluation approaches for a class of parallel machine problems. Such approaches generalize the ideas presented in Ergun and Orlin (2006) and Liao et al. (2012) for single machine total weighted tardiness problems. The algorithm was tested in benchmark instances of several E–T problems. The results obtained show that our unified heuristic is capable of producing high-quality solutions when compared to the best ones available in the literature that were obtained by specific methods.

We also provide an extensive annotated bibliography on the problems related to those considered in this work, where we not only indicate the approach(es) used in each publication, but we also point out the characteristics of the problem(s) considered. In addition, we classify the heuristic and exact methods in different categories so as to have a better idea of the popularity of each type of solution procedure.

The remainder of this work is organized as follows. Section 2 specifies the range of problems solved by our unified heuristic whose detailed description can be found in Sect. 4. Section 3 presents an annotated bibliography on E–T works related to the class of problems considered in this research. Computational experiments are reported in Sect. 5. Finally, Sect. 6 contains the concluding remarks.

2 Problems considered

In this section, we enumerate the class of E–T problems, including some particular cases, that our unified heuristic is capable of solving. We start by characterizing the general problem followed by a list of particular cases.

Let $J = \{1, \dots, n\}$ be a set of jobs to be scheduled on a set of unrelated parallel machines given by $M = \{1, \dots, m\}$. For each job $j \in J$, let p_j^k , d_j , r_j , w_j' and w_j be its processing time in machine $k \in M$, due date, release date, earliness penalty weight and tardiness penalty weight, respectively. Also, let s_{ij}^k be the setup time required before starting to process job $j \in J$ if j is scheduled immediately after job $i \in J$ in machine $k \in M$. The objective is to minimize $\sum w_j' E_j + w_j T_j$, where $E_j = \max\{d_j - C_j, 0\}$ and $T_j = \max\{C_j - d_j, 0\}$ are the earliness and tardiness of a job $j \in J$, respectively, that depends on its completion time C_j . Idle time is allowed to be inserted between two consecutive jobs. According to the notation suggested by Graham et al. (1979), this problem can be referred to as $R|r_j, s_{ij}^k|\sum w_j' E_j + w_j T_j$.

A large number of problems arise as a special case of the problem described above, including well-known single

Table 1 Problems considered

Single machine	Identical machines	Uniform machines	Unrelated machines
$1 \sum w_j T_j$	$P \sum w_j T_j$	$Q \sum w_j T_j$	$R \sum w_j T_j$
$1 r_j \sum w_j T_j$	$P r_j \sum w_j T_j$	$Q r_j \sum w_j T_j$	$R r_j \sum w_j T_j$
$1 s_{ij} \sum w_j T_j$	$P s_{ij} \sum w_j T_j$	$Q s_{ij}^k \sum w_j T_j$	$R s_{ij}^k \sum w_j T_j$
$1 r_j, s_{ij} \sum w_j T_j$	$P r_j, s_{ij} \sum w_j T_j$	$Q r_j, s_{ij}^k \sum w_j T_j$	$R r_j, s_{ij}^k \sum w_j T_j$
$1 \sum w'_j E_j + w_j T_j$	$P \sum w'_j E_j + w_j T_j$	$Q \sum w'_j E_j + w_j T_j$	$R \sum w'_j E_j + w_j T_j$
$1 r_j \sum w'_j E_j + w_j T_j$	$P r_j \sum w'_j E_j + w_j T_j$	$Q r_j \sum w'_j E_j + w_j T_j$	$R r_j \sum w'_j E_j + w_j T_j$
$1 s_{ij} \sum w'_j E_j + w_j T_j$	$P s_{ij} \sum w'_j E_j + w_j T_j$	$Q s_{ij}^k \sum w'_j E_j + w_j T_j$	$R s_{ij}^k \sum w'_j E_j + w_j T_j$
$1 r_j, s_{ij} \sum w'_j E_j + w_j T_j$	$P r_j, s_{ij} \sum w'_j E_j + w_j T_j$	$Q r_j, s_{ij}^k \sum w'_j E_j + w_j T_j$	$R r_j, s_{ij}^k \sum w'_j E_j + w_j T_j$
$1 r_j \sum w_j C_j$	$P \sum w_j C_j$	$Q \sum w_j C_j$	$R \sum w_j C_j$
$1 s_{ij} \sum w_j C_j$	$P r_j \sum w_j C_j$	$Q r_j \sum w_j C_j$	$R r_j \sum w_j C_j$
$1 r_j, s_{ij} \sum w_j C_j$	$P s_{ij} \sum w_j C_j$	$Q s_{ij}^k \sum w_j C_j$	$R s_{ij}^k \sum w_j C_j$
$1 r_j \sum w_j F_j$	$P r_j, s_{ij} \sum w_j C_j$	$Q r_j, s_{ij}^k \sum w_j C_j$	$R r_j, s_{ij}^k \sum w_j C_j$
$1 r_j, s_{ij} \sum w_j F_j$	$P r_j \sum w_j F_j$	$Q r_j \sum w_j F_j$	$R r_j \sum w_j F_j$
	$P r_j, s_{ij} \sum w_j F_j$	$Q r_j, s_{ij}^k \sum w_j F_j$	$R r_j, s_{ij}^k \sum w_j F_j$

machine problems without sequence-dependent setup times such as $1||\sum w_j T_j$, $1|r_j|\sum w_j T_j$, $1||\sum w'_j E_j + w_j T_j$, $1|r_j|\sum w'_j E_j + w_j T_j$, $1|r_j|\sum w_j C_j$ that can be efficiently solved by the exact algorithm of Tanaka and Fujikuma (2012) for instances with up to 200 jobs (sometimes even 300 jobs as in the case of $1||\sum w_j T_j$). Note that the latter problem, which consists of minimizing the average weighted completion time, is a particular case of problem $1|r_j|\sum w_j T_j$ when all due dates are admitted to be zero, i.e., $T_j = \max\{C_j - 0, 0\} = C_j, \forall j \in J$. Although the proposed heuristic is capable of dealing with these problems, its performance is simply not as good as some exact algorithms such as the one of Tanaka and Fujikuma (2012). In fact, it is really hard to devise heuristic algorithms with a superior or at least equivalent performance than the exact ones for these problems, even for large size instances. Our heuristic still finds the optimal solution for most of the existing instances, but not as fast as the state-of-the-art exact methods.

Another type of objective function that arise as particular case of $\sum w_j T_j$ is when one aims at minimizing the average weighted flow time, which is given by $\sum w_j F_j = \sum w_j (C_j - r_j)$. Note that by setting $d_j = r_j, \forall j \in J$, we obtain $T_j = \max\{C_j - r_j, 0\} = F_j, \forall j \in J$.

Table 1 lists some of the main problems that appear as special case of problem $R|r_j, s_{ij}^k|\sum w'_j E_j + w_j T_j$ and where our heuristic can be applied, including those mentioned above for the sake of completeness. Problems like $1||w_j C_j$ do not appear in the table because we only considered $\mathcal{NP}$ -hard problems. We also do not explicitly include problems with unitary weights because they are simply particular cases of the weighted problems. For example, problem $1|s_{ij}|\sum T_j$ is well studied in the literature, but in principle any algo-

rithm developed for problem $1|s_{ij}|\sum w_j T_j$ can be used to solve the former one. In fact, some recent works on problem $1|s_{ij}|\sum w_j T_j$ (Kirlik and Oğuz 2012; Tanaka and Araki 2013a; Subramanian et al. 2014; Xu et al. 2014; Subramanian and Farias 2017) also considered the version with unitary weights.

We did not perform computational experiments for all problems, not only for the reasons mentioned above, but also due to lack of publicly available instances. In addition, for some particular problems in which there are available instances, the authors who proposed them did not report lower/upper bounds.

3 An annotated bibliography for E–T problems related to those considered in this work

There is a vast literature on (E–)T scheduling. The first publications started to appear in the 1980s and they were surveyed by Baker and Scudder (1990). Another survey was later performed by Lauff and Werner (2004) for E–T problems in a multi-machine environment. Recently, Ratli et al. (2013) presented an overview on mathematical formulations and heuristics proposed for E–T problems but on a single machine environment. In these three works, the authors only reviewed problems with common due dates for all jobs. An entire book devoted to E–T scheduling was written by Józefowska (2007) where the author compiled the main models and algorithms for this class of problems, including those with common and individual due dates, respectively. The latter case is not only more general but also considered to be more challenging, since the first one has some properties that facilitates schedul-

ing decisions. For example, it is known that idle time does not appear in the optimal solution of some variants where common due dates are considered, which makes the problem easier to be solved.

Moreover, problems with other characteristics such as learning effects, preemption, maintenance, deteriorating jobs are not considered in this section. We also do not consider approximation algorithms for problems that aim at minimizing either the total weighted completion time (in this case we refer the reader to the survey of [Chekuri and Khanna 2004](#)) or the total flow time (see [Kellerer et al. 1996](#); [Leonardi and Raz 2007](#)). In addition, we only list the works published in the last 25 years. Finally, despite all our efforts in performing a complete enumeration of the large related literature, we cannot ensure that the annotated bibliography presented in this section include all the available relevant work.

3.1 Single machine environment

A considerable amount of the research involving (E-)T problems in the literature consider a single machine environment. Since there is a very large number of published works, it becomes rather impractical (and also it is beyond our goal) to provide a detailed description of each of them. Instead, we summarize, in chronological order, most of these works in Table 2, where we specify the solution approach; the scheduling characteristics such as the existence of sequence-dependent setup times (s_{ij}), release dates (r_j) and idle time (IT); and the type of objective considered by each method, namely (non-)weighted tardiness (T_j), (non-)weighted earliness and tardiness ($E_j + T_j$), (non-)weighted flow time (F_j) and (non-)weighted completion time (C_j).

From Table 2, we can observe that the wide range of heuristic procedures for single machine problems proposed in the literature can be classified as follows:

- Constructive or improvement heuristics ([Raman et al. 1989](#); [Potts and Van Wassenhove 1991](#); [Yano and Kim 1991](#); [Della Croce 1995](#); [Li 1997](#); [Lee et al. 1997](#); [Cicirello and Smith 2005](#); [Mason et al. 2005](#); [Sourd 2005](#); [Ying et al. 2009](#); [Yoon and Lee 2011](#)).
- Efficient local search procedures ([Congram et al. 2002](#); [Grosso et al. 2004](#); [Ergun and Orlin 2006](#); [Hendel and Sourd 2006](#); [Sourd 2006](#); [Kedad-Sidhoum and Sourd 2010](#); [Liao et al. 2012](#); [Subramanian and Farias 2017](#)).
- Population-based metaheuristics:
 - Genetic Algorithms (GAs) ([Rubin and Ragatz 1995](#); [Cicirello 2006](#); [Lin and Ying 2007](#); [Tsai 2007](#); [Sioud et al. 2012](#)).
 - Evolutionary Algorithms (EAs) ([Feldmann and Biskup 2003](#); [Tasgetiren et al. 2009](#)).
 - Memetic Algorithms (MAs) ([França et al. 2001](#)).

- Ant Colony Optimization (ACO) ([Gagné et al. 2002](#); [Liao and Juan 2007](#); [Anghinolfi and Paolucci 2008](#); [Mandahawi et al. 2011](#)).
- Particle Swarm Optimization (PSO) ([Anghinolfi and Paolucci 2009](#)).
- Scatter Search (SS) ([Bozejko 2010](#); [Guo and Tang 2015](#)).
- Local search-based metaheuristics:
 - Simulated Annealing (SA) ([Potts and Van Wassenhove 1991](#); [Tan and Narasimhan 1997](#); [Feldmann and Biskup 2003](#); [Lin and Ying 2007](#)).
 - Tabu Search (TS) ([Wan and Yen 2002](#); [Lin and Ying 2007](#)).
 - Variable Neighborhood Search (VNS) ([Kirlik and Oğuz 2012](#)).
 - Iterated Local Search (ILS) ([Congram et al. 2002](#); [Ying et al. 2009](#); [Kedad-Sidhoum and Sourd 2010](#); [Xu et al. 2013](#); [Subramanian et al. 2014](#)).
 - Greedy Randomized Adaptive Search Procedure (GRASP) ([Gupta and Smith 2006](#)).
- Hybrid metaheuristics ([Gagné et al. 2005](#); [M'Hallah 2007](#); [Liao and Cheng 2007](#); [Arroyo et al. 2009](#); [Sioud et al. 2012](#)).
- Mathematical Programming-based heuristics ([Avella et al. 2005](#); [Bülbul et al. 2007](#)).
- Alternative methods such as Beam Search (BS) ([Esteve et al. 2006](#); [Valente and Alves 2008](#)), Spaced-Based Local Search (SBLS) and Path-Relinking (PR) ([Gupta and Smith 2006](#)).

As for the exact methods, they mostly consist of a partial combination between Dynamic Programming (DP), Lagrangian Relaxation (LR), Branch-and-Bound (B&B) and Mixed Integer Programming (MIP) formulations, and they can be categorized as follows:

- B&B based on domination rules ([Belouadah et al. 1992](#); [Chu 1992a, b](#); [Selim Akturk and Ozdemir 2000](#); [Ronconi and Kawamura 2010](#)).
- DP + LR, such as the Successive Sublimation Dynamic Programming (SSDP) algorithm of Tanaka et al. ([Tanaka and Fujikuma 2008](#); [Tanaka et al. 2009](#); [Tanaka 2012](#); [Tanaka and Fujikuma 2012](#); [Tanaka and Araki 2013a](#)).
- B&B + LR, such as the methods of [Sourd and Kedad-Sidhoum \(2003\)](#), [Sourd \(2005\)](#), [Sourd and Kedad-Sidhoum \(2008\)](#), [Sourd \(2009\)](#) and [Li \(1997\)](#).
- B&B + DP + Constraint Programming (CP) ([Pan and Shi 2008](#)).
- B&B over linear programming (LP) relaxations based on Column Generation (CG) and/or cutting planes ([Van den Akker et al. 2000](#); [Bigras et al. 2008](#)).

Table 2 Single machine environment—list of works

Reference	Approach(es)	s_{ij}	r_j	T_j	$E_j + T_j$	C_j	F_j	IT
Raman et al. (1989)	Constructive Heuristic	✓		✓				
Dyer and Wolsey (1990)	Formulations and valid inequalities		✓			✓		✓
Potts and Van Wassenhove (1991)	SA, Constructive and randomized interchanging heuristics			✓				✓
Yano and Kim (1991)	Heuristics, DP				✓			✓
Belouadah et al. (1992)	B&B		✓			✓		✓
Chu (1992a)	B&B		✓				✓	
Chu (1992b)	B&B		✓	✓				
Rubin and Ragatz (1995)	GA	✓		✓				
Della Croce (1995)	Local search, Lower bounds		✓	✓	✓	✓		
Tan and Narasimhan (1997)	SA	✓		✓				
Li (1997)	B&B + LR, Heuristic				✓			
Lee et al. (1997)	Constructive Heuristic	✓		✓				
Van den Akker et al. (2000)	Time-indexed formulation, CG		✓			✓	✓	✓
Selim Akturk and Ozdemir (2000)	B&B		✓	✓				
França et al. (2001)	GA, MA	✓		✓				
Wan and Yen (2002)	TS				✓			✓
Gagné et al. (2002)	ACO	✓		✓				
Congram et al. (2002)	Dynasearch, Iterated dynasearch, ILS			✓				
Feldmann and Biskup (2003)	EA, SA, Threshold Accepting				✓			
Sourd and Kedad-Sidhoum (2003)	B&B + LR				✓			✓
Guo et al. (2004)	Experimental analysis with an approximation algorithm		✓				✓	
Grosso et al. (2004)	Dynasearch, Eliminations rules			✓				
Sourd (2005)	Time-indexed formulation, B&B + LR, Multi-start heuristic	✓			✓			✓
Gagné et al. (2005)	TS + VNS	✓		✓				
Cicirello and Smith (2005)	Heuristics	✓		✓				
Mason et al. (2005)	Moving block heuristic				✓			✓
Avella et al. (2005)	Lagrangian heuristic		✓			✓	✓	✓
Ergun and Orlin (2006)	Fast neighborhood search			✓				
Esteve et al. (2006)	Beam search		✓		✓			✓
Cicirello (2006)	GA	✓		✓				
Gupta and Smith (2006)	GRASP, PR, Problem Spaced-Based Local Search	✓		✓				
Hendel and Sourd (2006)	Efficient local search				✓			✓
Sourd (2006)	Dynasearch	✓	✓		✓			✓
Bülbüç et al. (2007)	Preemption-based relaxation + Heuristic		✓		✓			✓
M'Hallah (2007)	GA + Hill-Climbing + SA				✓			
Liao and Juan (2007)	ACO	✓		✓				
Liao and Cheng (2007)	VNS + TS				✓			
Lin and Ying (2007)	GA, SA, TS	✓		✓				
Tsai (2007)	GA		✓		✓			✓
Pan and Shi (2008)	Hybrid approach: B&B + DP + CP		✓			✓		✓
Tanaka and Fujikuma (2008)	SSDP		✓		✓			✓
Anghinolfi and Paolucci (2008)	ACO	✓		✓				
Sourd and Kedad-Sidhoum (2008)	B&B + LR		✓		✓			✓

Table 2 continued

Reference	Approach(es)	s_{ij}	r_j	T_j	$E_j + T_j$	C_j	F_j	IT
Valente and Alves (2008)	Beam Search	✓		✓				
Bigras et al. (2008)	Formulations, B&B	✓		✓				
Anghinolfi and Paolucci (2009)	PSO	✓		✓				
Tasgetiren et al. (2009)	EA	✓		✓				
Arroyo et al. (2009)	ILS + GRASP	✓		✓				
Ying et al. (2009)	Iterated Greedy algorithm	✓		✓				
Sourd (2009)	B&B + LR				✓			✓
Tanaka et al. (2009)	SSDP			✓	✓			
Geiger (2010)	Empirical Analysis	✓		✓				
Bozejko (2010)	Parallel SS	✓		✓				
Kedad-Sidhoum and Sourd (2010)	ILS, Fast Neighborhood Search				✓			✓
Ronconi and Kawamura (2010)	B&B				✓			
Yoon and Lee (2011)	Constructive heuristics			✓				
Mandahawi et al. (2011)	ACO	✓		✓				
Kirlik and Oğuz (2012)	VNS	✓		✓				
Liao et al. (2012)	Fast neighborhood Search	✓		✓				
Sioud et al. (2012)	Hybrid GA	✓		✓				
Tanaka and Fujikuma (2012)	SSDP		✓	✓	✓	✓		✓
Tanaka (2012)	SSDP		✓		✓			✓
Sioud et al. (2012)	GA + ACO	✓		✓				
Wan and Yuan (2013)	Proof of strongly $\mathcal{NP}$ -Hardness for problem 1 $E_j + T_j$				✓			✓
Tanaka and Araki (2013a)	SSDP	✓		✓				
Xu et al. (2013)	ILS	✓		✓				
Subramanian et al. (2014)	ILS	✓		✓				
Deng and Gu (2014)	ILS	✓		✓				
Guo and Tang (2015)	SS	✓		✓				
Subramanian and Farias (2017)	Efficient local search limitation strategy	✓		✓				

3.2 Parallel machine environment

In this section we list the related works that considered parallel machines. Tables 3, 4 and 5 summarize these works according to the environment, namely identical, uniform and unrelated parallel machine.

The heuristic approaches proposed for parallel machine problems can be classified as follows:

- Constructive and/or improvement heuristics (Lee and Pinedo 1997; Bank and Werner 2001; Weng et al. 2001; Sun and Wang 2003; Feng and Lau 2008; Biskup et al. 2008; Kedad-Sidhoum et al. 2008; Pfund et al. 2008; Mason et al. 2009; Xi et al. 2015).
- Population-based metaheuristics:
 - GA (Sivrikaya-Şerifoğlu and Ulusoy 1999; Vallada and Ruiz 2012; Amorim et al. 2013; Amorim 2013; Schaller 2014).
 - EA (Yousefi and Yusuff 2013; Lin and Hsieh 2014).
 - ACO (Zhou et al. 2007).
- Local search-based metaheuristics:
 - SA (Guinet 1995; Lee and Pinedo 1997; Koulamas 1997; Radhakrishnan and Ventura 2000; Bank and Werner 2001).
 - TS (Eom et al. 2002; Bilge et al. 2004; Kim et al. 2006; Logendran et al. 2007; Lee et al. 2013; Schaller 2014).
 - ILS (Rodrigues et al. 2008; Amorim 2013).
 - GRASP (Armentano and de França Filho 2007).
 - Large Neighborhood Search (LNS) (Rios-Solis and Sourd 2008).
- Hybrid metaheuristics (Anghinolfi and Paolucci 2007; Della Croce et al. 2012; M'Hallah and Al-Khamis 2012; Nogueira et al. 2014).

Table 3 Identical parallel machine environment—list of works

Reference	Approach(es)	s_{ij}	r_j	T_j	$E_j + T_j$	C_j	F_j	IT
Webster (1992)	Lower bounds		✓				✓	
Webster (1993)	Optimal priority rules for a special case of problem $P r_j \sum w_j F_j$		✓				✓	
Webster (1995)	Lower bounds		✓				✓	
Belouadah and Potts (1994)	B&B + LR					✓		
Lee and Pinedo (1997)	Preprocessing phase + Apparent tardiness cost with setups heuristic (ATCS) + SA	✓		✓				
Koulamas (1997)	Lower bounds, SA			✓				
Azizoglu and Kirca (1998)	B&B			✓				
Azizoglu and Kirca (1999)	B&B					✓		
Chen and Powell (1999)	CG					✓		
Radhakrishnan and Ventura (2000)	SA	✓			✓			
Eom et al. (2002)	EDD + ATCS + TS	✓		✓				
Yalaoui and Chu (2002)	B&B			✓				
Sun and Wang (2003)	DP, Constructive heuristic				✓			
Kim et al. (2006)	TS	✓	✓	✓				✓
Omar and Teo (2006)	MIP Formulation	✓			✓			✓
Yalaoui and Chu (2006)	B&B		✓			✓		✓
Shim and Kim (2007b)	B&B			✓				
Kedad-Sidhoum et al. (2008)	Time-indexed formulation, LR, CG, Efficient local search		✓		✓			✓
Feng and Lau (2008)	Squeaky Wheel Optimization (SWO)	✓			✓			✓
Rios-Solis and Sourd (2008)	Exponential Neighborhood Search				✓			
Pfund et al. (2008)	Apparent tardiness cost with setups and ready times (ATCSR) heuristic	✓	✓	✓				✓
Nessah et al. (2008)	B&B		✓			✓		✓
Biskup et al. (2008)	Constructive Heuristic			✓				
Rodrigues et al. (2008)	ILS			✓				
Tanaka and Araki (2008)	B&B + LR			✓				
Baptiste et al. (2008)	Lower Bounds		✓	✓		✓		✓
Mason et al. (2009)	Moving block heuristic				✓			✓
Pessoa et al. (2010)	Time-indexed formulation, Branch-cut-and-price			✓				
Jouglet and Savourey (2011)	B&B		✓	✓				✓
M'Hallah and Al-Khamis (2012)	MIP, Hybrid heuristic (steepest descent + GA + SA)				✓			✓
Della Croce et al. (2012)	ILS + Very Large Neighborhood Search			✓				
Amorim et al. (2013)	Hybrid GA				✓			
Amorim (2013)	GA, ILS, PR				✓			
Schaller (2014)	TSs, GAs, B&B	✓		✓				
Xi et al. (2015)	Look-ahead constructive heuristic	✓	✓	✓				✓

Table 4 Uniform parallel machine environment—list of works

Reference	Approach(es)	s_{ij}^k	r_j	T_j	$E_j + T_j$	C_j	F_j	IT
Webster (1992)	Lower bounds		✓				✓	
Guinet (1995)	SA			✓				
Azizoglu and Kirca (1998)	B&B			✓				
Azizoglu and Kirca (1999)	B&B					✓		
Chen and Powell (1999)	CG					✓		
Sivrikaya-Şerifoğlu and Ulusoy (1999)	GA	✓	✓		✓			✓
Balakrishnan et al. (1999)	Mathematical formulation + Benders' decomposition	✓	✓		✓			
Biskup and Feldmann (2001)	Benchmark instances					✓		
Bilge et al. (2004)	TS	✓	✓	✓				
Anghinolfi and Paolucci (2007)	TS + SA + VNS	✓	✓	✓				
Armentano and de França Filho (2007)	GRASP	✓	✓	✓				
Raja et al. (2008)	SA + Fuzzy Logic	✓			✓			
Yousefi and Yusuff (2013)	Imperialist Competitive Algorithm				✓			✓
Lin (2013)	Models/formulations					✓		
Li et al. (2014)	Agent-based algorithm + Lower Bounds		✓			✓		✓

Table 5 Unrelated parallel machine environment—list of works

Reference	Approach(es)	s_{ij}^k	r_j	T_j	$E_j + T_j$	C_j	F_j	IT
Webster (1992)	Lower bounds		✓				✓	
Zhu and Heady (2000)	MIP formulation	✓			✓			
Bank and Werner (2001)	Constructive + iterative heuristics, SA		✓		✓			✓
Weng et al. (2001)	Constructive heuristics	✓				✓		
Liaw et al. (2003)	B&B			✓				
Zhou et al. (2007)	ACO			✓				
Shim and Kim (2007a)	B&B			✓				
Logendran et al. (2007)	Six algorithms based on TS	✓	✓	✓				
Pereira-Lopes and Valério de Carvalho (2007)	Branch-and-price	✓	✓	✓				✓
Akyol and Bayhan (2008)	Neural network	✓			✓			
Li and lin Yang (2009)	Survey	✓	✓		✓			
Vallada and Ruiz (2012)	MIP formulation, GA	✓			✓			✓
Lee et al. (2013)	TS	✓		✓				
Polyakovskiy and M'Hallah (2014)	Multi-Agent System Heuristic				✓			✓
Nogueira et al. (2014)	GRASP + PR + ILS	✓			✓			✓
Lin and Hsieh (2014)	Modified ATCSR and Electromagnetism-like Algorithm (EMA)	✓	✓	✓				
Bülbul and Şen (2017)	Preemptive relaxation + Benders' decomposition					✓		
Şen and Bülbul (2015)	Preemptive relaxation + Benders' decomposition + solver SiPS/SiPSi (Tanaka et al. 2009) for obtaining upper bounds		✓	✓	✓			✓

- Artificial Intelligence-based heuristics (Akyol and Bayhan 2008; Raja et al. 2008; Polyakovskiy and M'Hallah 2014; Li et al. 2014).
- Mathematical Programming-based heuristics (Kedad-Sidhoum et al. 2008; Şen and Bülbül 2015).

The exact methods developed for parallel machine problems can be classified as follows:

- B&B based on domination rules (Azizoglu and Kirca 1998, 1999; Yalaoui and Chu 2002; Liaw et al. 2003; Yalaoui and Chu 2006; Shim and Kim 2007a, b; Jouglet and Savourey 2011; Nessah et al. 2008; Schaller 2014).
- B&B + LR (Belouadah and Potts 1994; Tanaka and Araki 2008).
- B&B over linear programming (LP) relaxations based on CG and/or cutting planes (Chen and Powell 1999; Pereira-Lopes and Valério de Carvalho 2007; Pessoa et al. 2010).
- Compact MIP formulations (Balakrishnan et al. 1999; Zhu and Heady 2000; Omar and Teo 2006; Vallada and Ruiz 2012; Lin 2013).
- Preemption-based relaxation (Bülbül and Şen 2017).

3.3 Discussion

3.3.1 Exact algorithms

When analyzing the scalability of the best exact algorithms for (E–)T scheduling problems, it appears that their efficiency is highly sensitive to the presence of setup times and multiple machines. For example, while the SSDP algorithm of Tanaka et al. (2009) could solve single machine instances (considering different objective functions but no idle times) without setup times with up to 300 jobs, the SSDP algorithm of Tanaka and Araki (2013b) for problem $1|s_{ij}|\sum w_j T_j$, which is the best known for this variant, sometimes struggled to solved instances with up to 60 jobs to optimality. Moreover, the Lagrangian relaxation-based algorithm of Tanaka and Araki (2008) could only solve instances with up to 25 jobs of problem $P||\sum T_j$. Pessoa et al. (2010), however, managed to solve large instances with up to 100 jobs for problem $P||\sum w_j T_j$ with their branch-cut-and-price algorithm over an arc-time-indexed formulation for the same problem.

Idle times also seem to affect the performance of exact algorithms, but not substantially as the presence of setup times and multiple machines. For instance, the SSDP algorithm proposed by Tanaka and Fujikuma (2012) for solving a class of single machine (E–)T scheduling problems with idle time could solve instances with to 200 jobs to optimality.

3.3.2 Heuristic algorithms

From the list of works mentioned in Sects. 3.1 and 3.2, we were not able to identify the existence of general purpose heuristics for single/parallel machine (E–)T scheduling problems, as opposed, for example, to the field of vehicle routing, where there has been a trend in the development of efficient unified heuristics (Cordeau et al. 2001; Røpke and Pisinger 2006; Pisinger and Røpke 2007; Cordeau and Maischberger 2012; Subramanian et al. 2013; Vidal et al. 2013, 2014; Derigs and Vogel 2014).

In contrast to the exact algorithms for (E–)T scheduling problems, it is quite hard to point out which are the best known heuristics for such problems. This is partially because there are very few consolidated benchmark instances in the literature and many authors often test their proposed methods in instances generated in their own work. Nevertheless, there are few exceptions in which this does not happen. For example, there are benchmark instances publicly available for problems $1|s_{ij}|\sum T_j$ and $1|s_{ij}|\sum w_j T_j$. In such cases, a comparison is possible to be performed and the best known heuristics are those of Xu et al. (2013, 2014), Subramanian et al. (2014), and Subramanian and Farias (2017).

Due to the drawbacks mentioned above, we do not intend to identify the best heuristic algorithms for (E–)T scheduling problems. Instead, we will discuss the challenges in developing efficient heuristics, especially those based in local search, for this class of problems.

It is known that the existence of idle times can significantly affect the performance of a heuristic algorithm, even more than the presence of setup times and multiple machines. In fact, we are not aware of any heuristic method whose overall performance is comparable to the exact algorithms proposed in Tanaka et al. (2009), Tanaka and Fujikuma (2012) for single machine problems that consider idle time such as $1||\sum w'_j E_j + w_j T_j$ and $1|r_j|\sum w'_j E_j + w_j T_j$, at least for instances involving up to 300 jobs. This somewhat explains why most of the early work on E–T considered common due dates for the jobs (Baker and Scudder 1990), where the optimal solutions do not include idle time. Other works that dealt with E–T objective functions simply disregard the idle times when solving the problems (Radhakrishnan and Ventura 2000; Sun and Wang 2003; Rios-Solis and Sourd 2008; Raja et al. 2008; Amorim et al. 2013; Amorim 2013). The challenge of solving variants that include this feature is that one should solve a subproblem known as *timing problem* (Garey et al. 1988; Vidal et al. 2015b) that consists of determining the optimal start times of the jobs given a sequence, which can be solved in $\mathcal{O}(n_k \log n_k)$ (Hendel and Sourd 2007). This is usually the bottleneck of local search-based heuristics, because it is hard to efficiently compute the cost of a move. Yet, there are some procedures such as those proposed by Hendel and Sourd (2006) that efficiently deal with

this issue when there is only a single machine and setup times are not considered. A more general and efficient procedure that considers parallel machines and setup times was developed by Ibaraki et al. (2005, 2008).

In problems without idle time, it is generally easier to evaluate the cost of a solution. In fact, there are some move evaluation schemes that can compute the cost of a neighbor solution in amortized $\mathcal{O}(1)$ time (Ergun and Orlin 2006), even when setup times are considered (Liao et al. 2012) (though in practice this task is usually performed in linear time). We discuss different move evaluation schemes in Sects. 4.2.2, 4.2.3, 4.2.4 and 4.2.5.

As a matter of fact, parts of a heuristic algorithm that were not designed to allow idle time should be substantially redesigned to address this point. Note that the opposite can also happen, that is, a heuristic designed to handle idle time may turn out to have a slow performance when considering instances without idle time. This represents a substantial challenge when it comes to developing a general purpose heuristic.

4 The unified heuristic algorithm

The proposed unified heuristic, called UILS, is mostly based on ILS (Lourenço et al. 2002). This metaheuristic basically alternates between intensification (local search) and diversification (perturbation) procedures in order to escape from local optima. We modified the original ILS algorithm by allowing multiple restarts of the method. Previous works showed that this type of implementation, along with a Randomized Variable Neighborhood Descent (RVND) approach in the local search phase, yielded high-quality results for different kinds of problems (Subramanian et al. 2010, 2014; Silva et al. 2012; Penna et al. 2013; Subramanian and Battarra 2013; Martinelli et al. 2013; Vidal et al. 2015a), most of them in the field of routing, including the unified algorithms presented in Subramanian (2012), Subramanian et al. (2013).

Given the previous successful implementations of the multi-start ILS-RVND algorithm, we decided to extend this method to E-T scheduling problems. However, as pointed out in Sect. 1, several issues arise when dealing with scheduling problems that usually do not appear in vehicle routing problems. The main adaptation was in the local search phase where we implemented a tailored move evaluation approach according to the characteristics of the problem, such as the existence or not of earliness penalties, idle time, release dates and sequence-dependent setup times. This is one of the key aspects for the versatility and potential scalability of the proposed heuristic when facing problems with distinct features. The flowchart of UILS is found in Fig. 1.

The multi-start heuristic starts by generating an initial solution using a very simple greedy randomized or a com-

pletely randomized insertion procedure. Next, a local search is performed using RVND, but with a specific move evaluation scheme that depends on the characteristics of the problem. On the one hand, when idle times are not considered, the choice of the move evaluation scheme to be used depends on the existence of setup times. On the other hand, when idle times are taken into account, the decision on the move evaluation scheme to be used depends on the presence of earliness penalties. Note that in this latter case, the existence of setup times does not affect such decision. Moreover, if the maximum number of consecutive perturbations without improvements (I_{ILS}) is not achieved (ILS stopping criterion), then the algorithm modifies the incumbent solution of the current multi-start iteration by applying a perturbation mechanism and then it restarts the local search procedure from that perturbed solution. Otherwise, the algorithm restarts from the beginning. If the maximum number of restarts (I_R) is achieved then the heuristic stops and returns the best solution found.

4.1 Initial solutions

For problems involving multiple machines, the initial solutions are generated at random, whereas for single machine problems without release date the solutions are built either at random or following the earliest due date (EDD) method.

When release dates are considered, the initial solutions are generated using the earliest release date (ERD) criterion. For the sake of diversification, EDD and ERD were implemented based on the greedy randomized approach of the constructive phase of GRASP (Feo and Resende 1995).

It is worth mentioning that, based on preliminary experiments, the use of more sophisticated methods for building initial solutions do not significantly affect, on average, neither the quality of the final solution, nor the speed of convergence toward a local optimum. Hence, for simplicity, we decided to keep the constructive procedure as simple as possible.

4.2 Local search

The local search is performed by a RVND procedure (Mladenović and Hansen 1997; Subramanian 2012), which consists of randomly choosing an unexplored neighborhood to proceed with the search every time another one fails to find an improved solution. Otherwise, all neighborhoods are allowed to be selected.

In what follows, we describe the neighborhood structures used in RVND, as well as the move evaluation procedures which can vary depending on the characteristics of the problem.

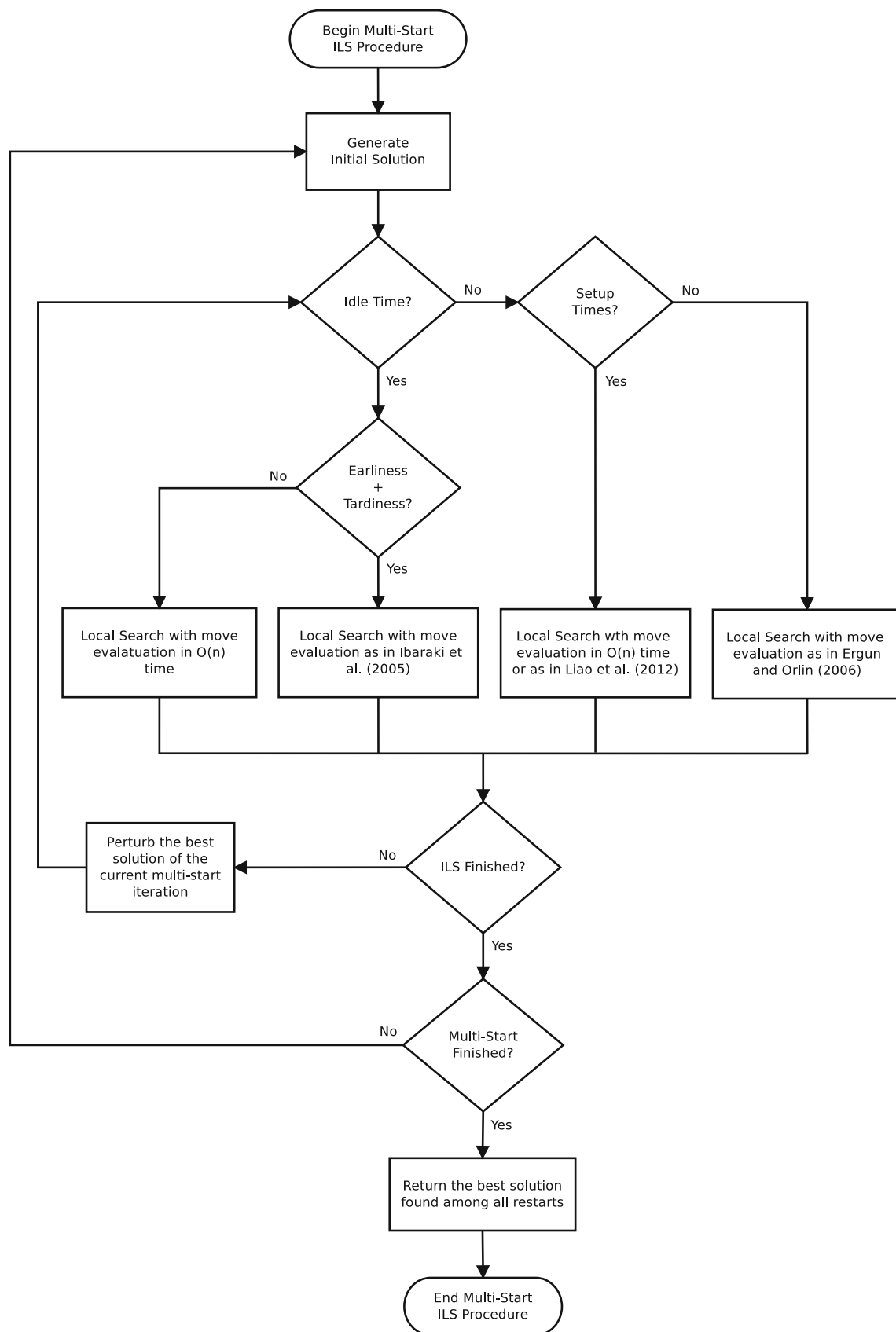


Fig. 1 Flowchart of the unified heuristic

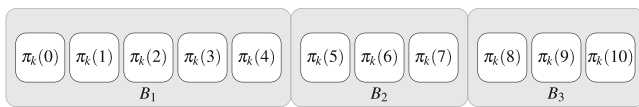


Fig. 2 Example of block representation

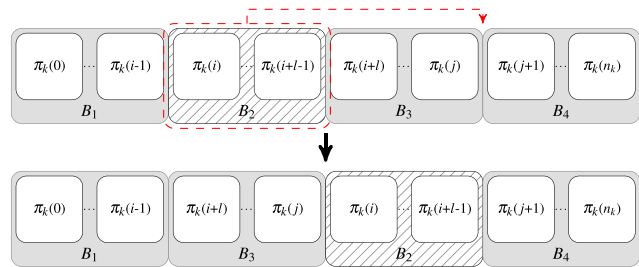


Fig. 3 l -block insertion forward intra-machine

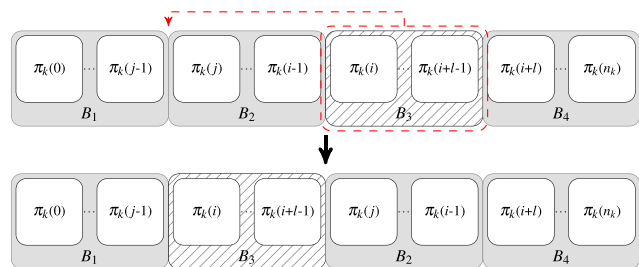


Fig. 4 l -block insertion backward intra-machine

4.2.1 Neighborhood structures

With a view of better illustrating the neighborhood structures used in our algorithm we resort to a block representation, which is commonly adopted in the scheduling literature. Let $\pi_k = (\pi_k(0), \pi_k(1), \dots, \pi_k(n_k))$ be a sequence of $n_k + 1$ jobs associated with a machine $k \in M$, with $\pi_k(0) = 0$ as a dummy job that is necessary for considering an eventual setup $s_{0\pi_k(1)}^k$ for processing the initial job of the sequence. A block B consists of subsequence of consecutive jobs. An example involving 10 jobs divided into three blocks ($B_1 = (\pi_k(0), \pi_k(1), \pi_k(2), \pi_k(3), \pi_k(4))$, $B_2 = (\pi_k(5), \pi_k(6), \pi_k(7))$, $B_3 = (\pi_k(8), \pi_k(9), \pi_k(10))$) is shown in Fig. 2.

The neighborhoods used in RVND are based on insertion and swap moves involving subsequences of jobs (blocks) and they are described next.

- l -block insertion intra-machine—consists of moving (reinserting) a block of size l , starting from job $\pi_k(i)$, to the position after job $\pi_k(j)$ in the same machine. A block (B_2) is moved forward (after B_3) when $i = 1, \dots, n_k - l$ and $j = i + l, \dots, n_k$, if $i < j$; and moved backward (B_3 before B_2) when $i = 2, \dots, n_k + 1 - l$ and $j = 0, \dots, n_k - l$, if $i > j$, as depicted in Figs. 3 and 4, respectively.

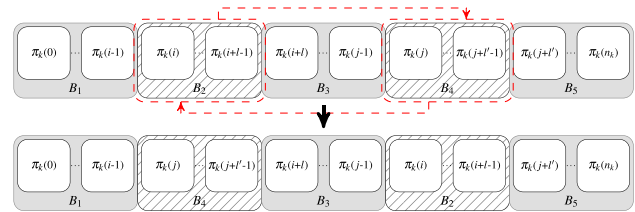


Fig. 5 (l, l') -block swap intra-machine

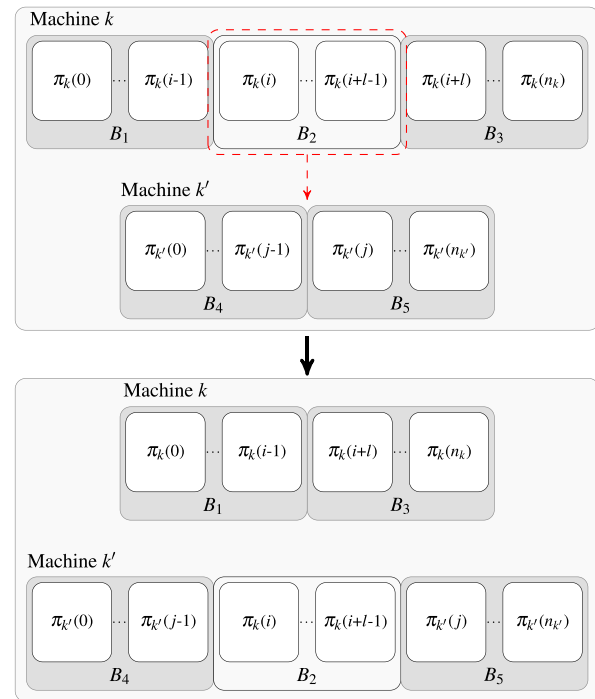


Fig. 6 l -block insertion inter-machine

- (l, l') -block swap intra-machine—consists of interchanging a block (B_2) of size l , starting from job $\pi_k(i)$, with another one belonging to the same machine (B_4) of size l' , starting from job $\pi_k(j)$, as shown in Fig. 5.
- l -block insertion inter-machine—consists of removing a block (B_2) of size l from a machine k , starting from $\pi_k(i)$, and inserting it in machine k' at position j , after block B_4 , as illustrated in Fig. 6.
- (l, l') -block swap inter-machine—consists of interchanging a block (B_2) of size l of a machine k , starting from job $\pi_k(i)$, with a block (B_5) of size l' of a machine k' that starts from job $\pi_{k'}(j)$, as presented in Fig. 7.

The size(s) of the block(s) in each type of neighborhood is an input parameter. Let L_{intra} , L_{inter} , L'_{intra} and L'_{inter} be the set of possible block sizes for the neighborhoods l -block insert intra-machine, l -block insert inter-machine, (l, l') -block swap intra-machine and (l, l') -block swap inter-machine, respectively. For example, if $L_{\text{intra}} = \{1, 2, \dots, 7\}$,

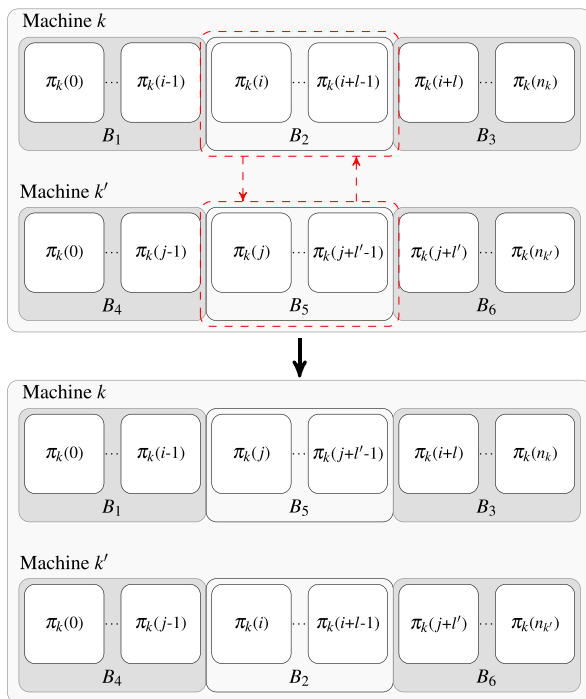


Fig. 7 (l, l') -block swap inter-machine

we assume that there are 7 neighborhoods of the type l -block insert intra-machine to be considered in the RVND procedure, each with a distinct value for l . Similarly, if $L'_{\text{inter}} = \{(1, 1), (2, 1), (2, 2), (1, 3), (2, 3), (3, 3)\}$, then the RVND procedure will consider 6 neighborhoods of the type (l, l') -block swap inter-machine, each of them with a different setting for (l, l') .

The size of each individual neighborhood of any of the four types mentioned above is $\mathcal{O}(n^2)$. For problems without idle time, a straightforward move evaluation can be performed in $\mathcal{O}(n)$ (Subramanian and Farias 2017), leading to an overall complexity of $\mathcal{O}(n^3)$ for enumerating and examining all possible moves from that neighborhood. However, when there are no sequence-dependent setup times, the move evaluation can be performed in amortized $\mathcal{O}(1)$ time by extending the ideas presented in Ergun and Orlin (2006), yielding an overall complexity of $\mathcal{O}(n^2)$. Otherwise, it is known that the move evaluation can still be performed in amortized $\mathcal{O}(1)$ time, but with an overall complexity of $\mathcal{O}(n^2 \log n)$, by extending the approach proposed in Liao et al. (2012). Nevertheless, it appears that this approach is only worth to be implemented in practice for large sized instances/sequences.

For problems with idle time, one should solve the timing problem, i.e., determine the optimal start of the jobs associated with the solution under evaluation, which is usually solved by dynamic programming (Ibaraki et al. 2005), and the resulting overall complexity is often greater than

Algorithm 1 CompCostBlock

```

1: procedure COMPCOSTBLOCK( $b', a, b, \pi_k, C_{b'}$ )
2:    $cost \leftarrow 0$ 
3:    $Ctemp \leftarrow C_{b'} + s_{\pi_k(b'), \pi_k(a)}^k + p_{\pi_k(a)}^k$   $\triangleright$  Global variable that stores
   a temporary completion time
4:   if  $Ctemp > d_{\pi_k(a)}$  then
5:      $cost \leftarrow w_{\pi_k(a)} \times (Ctemp - d_{\pi_k(a)})$ 
6:   else
7:      $cost \leftarrow w'_{\pi_k(a)} \times (d_{\pi_k(a)} - Ctemp)$ 
8:   for  $a' = a + 1 \dots b$  do
9:      $Ctemp \leftarrow Ctemp + s_{\pi_k(a'-1), \pi_k(a')}^k + p_{\pi_k(a')}^k$ 
10:    if  $Ctemp > d_{\pi_k(a')}$  then
11:       $cost \leftarrow cost + w_{\pi_k(a')} \times (Ctemp - d_{\pi_k(a')})$ 
12:    else
13:       $cost \leftarrow cost + w'_{\pi_k(a')} \times (d_{\pi_k(a')} - Ctemp)$ 
14:  return  $cost$ 

```

the case without idle time. However, when earliness penalties are not considered and the need for idle time is due to the existence of release dates, the timing problem becomes trivial and one should only ensure that the job does not start before its release date. This can be done in $\mathcal{O}(n)$ time and therefore the overall complexity in this case is $\mathcal{O}(n^3)$.

The next four sections present different ways of performing the move evaluation according to the characteristics of the problem.

4.2.2 Straightforward move evaluation for problems without idle time

Define a and b as the position associated with the first and last jobs of u -th block B_u in the sequence, respectively, and define b' as the position associated with the last job of the preceding block B_{u-1} in the sequence. Let $C_{b'}$ be the completion time of the job in position b' . Algorithm 1 shows how to compute the cost of block B_u in $\mathcal{O}(|B_u|)$ steps.

The cost of a neighbor solution can be computed by simply summing up the cost of each block involved in the move. For example, when evaluating the cost of a move related to the (l, l') -block swap intra-machine neighborhood, one can directly sum the costs of the blocks B_1, B_4, B_3, B_2 , and B_5 (see Fig. 5) in this particular order. Note that the cost of a block depends on the completion time of the last job of the previous sequence, as illustrated in Algorithm 1. In the worst case, the affected blocks may together contain all jobs of the instance, which imply in $\mathcal{O}(n)$ operations for computing the cost.

In order to speed up the process of computing the cost one can keep track of the cumulated cost up to a particular position of the sequence. Hence, define W_j^k as a data structure that stores the cumulated cost up to the position j of a sequence π_k , more precisely:

$$W_j^k = \sum_{i=1}^j c_i^k, \quad j = 1, \dots, n_k \quad (1)$$

where:

$$c_i^k = \begin{cases} w'_{\pi_k(i)}(d_{\pi_k(i)} - C_{\pi_k(i)}), & \text{if } C_{\pi_k(i)} \leq d_{\pi_k(i)} \\ w_{\pi_k(i)}(C_{\pi_k(i)} - d_{\pi_k(i)}), & \text{if } C_{\pi_k(i)} \geq d_{\pi_k(i)} \end{cases}$$

For example, the cost of block $B_1 = (\pi_k(0), \dots, \pi_k(5))$ can be accessed in $\mathcal{O}(1)$ time by directly verifying the value of W_5^k .

We remark that the straightforward way of computing the cost of a move described in this section is not new and we refer to [Subramanian and Farias \(2017\)](#) for a detailed description of the move evaluation using this type of approach. It is worthy of note that the same authors developed a local search limitation strategy, based on the setup variation due to a move, for problem $1||s_{ij}|\sum w_j T_j$ that turned out to be very efficient in practice. They implemented a filtering mechanism that avoids unpromising moves to be evaluated. We believe that this idea can be extended for problems involving multiple machines and sequence-dependent setup times.

4.2.3 Move evaluation for problems without both idle time and sequence-dependent setup times

[Ergun and Orlin \(2006\)](#) showed that a move of the 1-block swap intra-machine neighborhood can be evaluated in amortized $\mathcal{O}(1)$ time for problem $1||\sum w_j T_j$. This is done by storing information of the current solution (sequence) in auxiliary data structures that are updated in $\mathcal{O}(n^2)$ time, which in turn enable the move evaluation to be performed in constant time. In this work we extend their approach for the neighborhoods described in Sect. 4.2.1 to solve problem $R||\sum w'_j E_j + w_j T_j$ (without idle time) and its particular cases, that is, all problems without sequence-dependent setup times and release dates shown in Table 1.

Before describing the move evaluation schemes for all neighborhoods, we will introduce some useful auxiliary functions and data structures.

Let $\rho_j^k(t)$ be a nonnegative, convex and piecewise linear function that represents the penalty of start processing job $j \in J$ in machine $k \in M$ at time t :

$$\rho_j^k(t) = \begin{cases} M(r_j - t) + w'_j(d_j - p_j^k - t), & t \in [0, r_j] \\ w'_j(d_j - p_j^k - t), & t \in [r_j, d_j - p_j^k] \\ w_j(t - d_j + p_j^k), & t \in [d_j - p_j^k, +\infty) \end{cases}$$

where M is constant, positive and sufficiently large, used to penalize release date violation. For the ease of presentation we assume that $r_j \leq d_j - p_j^k$. If this condition is not satisfied

in practice, i.e., $r_j > d_j - p_j^k$, then the functions above should be modified accordingly.

Each function $\rho_j^k(t)$ is composed of three segments, where the transition points, a.k.a. *breakpoints*, are defined by r_j , d_j and p_j^k . Release dates were considered for the sake of generality, but their existence typically enforce the presence of idle times. Since we are not considering idle times in this section, we can simply disregard one of the segments by setting $r_j = 0, \forall j \in J$, and the functions will remain valid, but with two segments each.

Now, for a given sequence π_k , let $g_j^k(t)$ be a piecewise linear function denoting the cost of a block composed of jobs $\pi_k(j), \pi_k(j+1), \dots, \pi_k(n_k)$ sequenced in machine k in this order and that job $\pi_k(j)$ starts to be processed at time t . Note that n_k is the index of the last job assigned to machine k . Each function $g_j^k(t), \forall j \in J, \forall k \in M$, can be determined by the following recursion:

$$g_j^k(t) = \begin{cases} \rho_j^k(t), & j=n_k, \\ g_{j+1}^k(t + p_j^k + s_{j,j+1}^k) + \rho_j^k(t), & j \in (0, n_k) \end{cases} \quad t \in [0, +\infty)$$

To better clarify the meaning of functions $\rho_j^k(t)$ and $g_j^k(t)$, consider the example given in Table 6, but disregarding the release dates and setup times.

Now assuming, for instance, that $\pi_1 = (0, 1, 2)$, we can write functions $\rho_1^1(t), \rho_2^1(t), g_1^1(t)$ and $g_2^1(t)$ as follows:

$$\begin{aligned} \rho_1^1(t) &= \begin{cases} 10 - 2t, & t \in [0, 5] \\ 4(t - 5), & t \in [5, +\infty) \end{cases} \\ \rho_2^1(t) &= \begin{cases} 5 - 1t, & t \in [0, 5] \\ 2(t - 5), & t \in [5, +\infty) \end{cases} \\ g_2^1(t + p_1^1) &= \begin{cases} 2 - 1t, & t \in [0, 2] \\ 2(t - 2), & t \in [2, +\infty) \end{cases} \\ g_1^1(t) &= \begin{cases} 12 - 3t, & t \in [0, 2] \\ 6, & t \in [2, 5] \\ 6 + 6(t - 5), & t \in [5, +\infty) \end{cases} \end{aligned}$$

Figure 8 illustrates a graphical representation of the piecewise linear functions.

Functions $\rho_j^k(t)$ e $g_j^k(t)$ can be stored in memory by means of linked lists, here denoted as L_j_k , where each element of the list is associated with one of the segments of the piece-

Table 6 Example involving 5 jobs on a single machine

j	p_j^1	d_j	w'_j	w_j
1	3	8	2	4
2	2	7	1	2
3	1	4	2	4
4	3	3	1	3
5	1	13	1	1

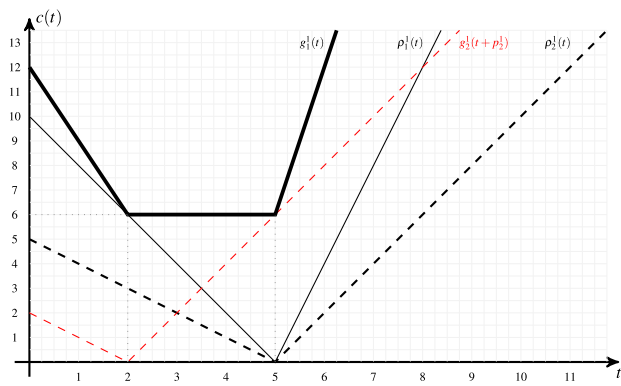
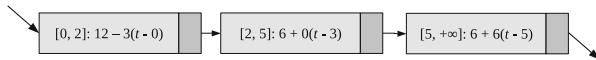


Fig. 8 Penalty functions

$$[b_1, b_2]: c - \alpha(t - b_1)$$

Fig. 9 Representation of a segment

Fig. 10 Representation of function $g_1^k(t)$ using a linked list

wise linear function. We use a data structure called *seg* to represent each segment of the function. Such data structure (see Fig. 9) stores the following piece of information:

- b_1 : beginning of the segment's interval;
- b_2 : end of the segment's interval;
- c : value of the function at time b_1 ;
- α : slope of the segment.

Figure 10 shows an example of how function $g_1^k(t)$ of Fig. 8 is stored in memory using the linked list L_{1_1} .

In order to compute the cost of a block $(\pi_k(j), \pi_k(j+1), \dots, \pi_k(n_k))$ starting at time t by means of functions $g_j^k(t)$ using the information mentioned above, one needs to walk through the breakpoints of the corresponding function until the interval $b_1 \leq t \leq b_2$ is reached and then compute the cost which is given by $c + \alpha \times (t - b_1)$.

Finally, another important data structure is the so-called *ProcessingList* (Ergun and Orlin 2006), which in our case is defined for all blocks of size l from a given sequence π_k . Each element of this list contains a pair of information, namely: *pos* and *p*, where the first stores the position of the first job of the block in the sequence, while the second stores the total processing time of the block in machine k' . The *ProcessingList* must be sorted in descending order according to the value of *p*. Algorithm 2 shows how the *ProcessingList* is created.

Algorithm 2 CreateProcessingList

```

1: procedure CREATEPROCESSINGLIST( $\pi_k, k', l$ )
2:    $p_B \leftarrow 0$   $\triangleright p_B$  is the total processing time of the block of size  $l$ 
3:   for  $i' = 1, \dots, l$  do
4:      $p_B \leftarrow p_B + p_{\pi_k(i')}$ 
5:   for  $i \leftarrow 1, \dots, n_k - l + 1$  do
6:     ProcessingList.pos[ $i$ ]  $\leftarrow i$ 
7:     ProcessingList.p[ $i$ ]  $\leftarrow p_B$ 
8:      $p_B \leftarrow p_B - p_{\pi_k(i)} + p_{\pi_k(i+l)}$ 
9:   sort(ProcessingList, p)
10:  return ProcessingList

```

Once the auxiliary functions and data structures were defined, it is now possible to show how to compute the cost of a sequence π'_k generated after applying one of the neighborhood operators.

For a given sequence π_k , $g_j^k(t)$ can have at most n_k breakpoints. Since $g_j^k(t)$ are piecewise linear functions, the complexity of obtaining the cost of a block $(\pi_k(j), \pi_k(j+1), \dots, \pi_k(n_k))$ starting at time t is, in principle, $\mathcal{O}(n_k)$, because in the worst case t can be greater than the last breakpoint of the function. Hence, the complexity of evaluating a move would be $\mathcal{O}(n_k)$. However, it is possible to evaluate a move of the neighborhoods in amortized $\mathcal{O}(1)$ time if the costs that depend on $g_j^k(t)$ are precomputed following a given order.

It is worth mentioning that an adaptation of the data structures proposed by Ergun and Orlin (2006) was already performed by Kedad-Sidhoum and Sourd (2010), where the authors considered the swap and insertion (intra-machine) neighborhoods on a single machine environment with E–T penalties. In contrast to Kedad-Sidhoum and Sourd (2010), our approach was designed to deal with inter-machine neighborhoods, more precisely, with the insertion and swap of blocks on parallel machine environments involving unrelated, uniform or identical machines with E–T penalties.

Table 7 shows how to compute the cost of the modified solution in amortized $\mathcal{O}(1)$ time given that the costs that depend on $g_j^k(t)$ are precomputed in $\mathcal{O}(n_k^2)$. The first column identifies the blocks related to the move using the same convention adopted in Figs. 3, 4, 5, 6 and 7, while the remaining ones show how to compute the cost of each particular block depending on the neighborhood and machines involved in the move. Also, as shown in the referred figures, we can observe that the number of resulting blocks of a move varies from 4 to 6, according to the neighborhood. For example, a move of the neighborhood l -block insertion inter considers blocks B_1 and B_3 on machine k , and blocks B_2 , B_4 and B_5 on machine k' . For a given solution, the indices i and j refer to the initial position of the blocks associated with the move (see Sect. 4.2.1 for more details). Hence, the cost of a neighbor solution can be computed by summing up the costs of the blocks involved in the move.

Table 7 Move evaluation schemes

Block	l -block insertion intra		$(l-l')$ -block swap intra		l -block insertion inter		$(l-l')$ -block swap inter	
	k		k		k	k'	k	k'
Complexity of performing a single move evaluation in amortized $\mathcal{O}(1)$ time								
B_1	W_{i-1}^k		W_{i-1}^k		W_{i-1}^k	–	W_{i-1}^k	–
B_2	$g_i^k(t_2) - g_{i+l}^k(t_3)$		$g_i^k(t_5) - g_{i+l}^k(t_6)$		–	$g_i^{k'}(t_{11}) - g_{i+l}^{k'}(t_{10})$	–	$g_i^{k'}(t_{11}) - g_{i+l}^{k'}(t_{10})$
B_3	$g_{i+l}^k(t_1) - g_{j+1}^k(t_2)$		$g_{i+l}^k(t_4) - g_j^k(t_5)$		$g_{i+l}^k(t_9)$	–	$g_{i+l}^k(t_{12})$	–
B_4	$W_{n_k}^k - W_j^k$		$g_j^k(t_7) - g_{j+l'}^k(t_8)$		–	$W_{j-1}^{k'}$	–	$W_{j-1}^{k'}$
B_5	–		$W_{n_k}^k - W_{j+l'-1}^k$		–	$g_j^{k'}(t_{10})$	$g_j^k(t_9) - g_{j+l'}^k(t_{12})$	–
B_6	–		–		–	–	–	$g_{j+l'}^{k'}(t_{10})$
Complexity of performing a single move evaluation in amortized $\mathcal{O}(l)/\mathcal{O}(\max\{l, l'\})$ time								
B_1	W_{i-1}^k		W_{i-1}^k		W_{i-1}^k	–	W_{i-1}^k	–
B_2	CompCostBlock(B_2)		CompCostBlock(B_2)		–	CompCostBlock(B_2)	–	CompCostBlock(B_2)
B_3	$g_{i+l}^k(t_1) - g_{j+1}^k(t_2)$		$g_{i+l}^k(t_4) - g_j^k(t_5)$		$g_{i+l}^k(t_9)$	–	$g_{i+l}^k(t_{12})$	–
B_4	$W_{n_k}^k - W_j^k$		CompCostBlock(B_4)		–	$W_{j-1}^{k'}$	–	$W_{j-1}^{k'}$
B_5	–		$W_{n_k}^k - W_{j+l'-1}^k$		–	$g_j^{k'}(t_{10})$	CompCostBlock(B_5)	–
B_6	–		–		–	–	–	$g_{j+l'}^{k'}(t_{10})$

Where:

$$t_1 = C_{i-1}^k; \quad t_2 = C_j^k - \sum_{a=i}^{i+l-1} p_{\pi_k(a)}^k; \quad t_3 = C_j^k; \quad t_4 = C_{i-1}^k + \sum_{a=j}^{j+l'-1} p_{\pi_k(a)}^k;$$

$$t_5 = C_{j+l'-1}^k - \sum_{a=i}^{i+l-1} p_{\pi_k(a)}^k; \quad t_6 = C_{j+l'-1}^k; \quad t_7 = C_{i-1}^k; \quad t_8 = C_{i-1}^k + \sum_{a=j}^{j+l'-1} p_{\pi_k(a)}^k;$$

$$t_9 = C_{i-1}^k; \quad t_{10} = C_{j-1}^{k'} + \sum_{a=i}^{i+l-1} p_{\pi_k(a)}^{k'}; \quad t_{11} = C_{j-1}^{k'}; \quad t_{12} = C_{i-1}^k + \sum_{a=j}^{j+l'-1} p_{\pi_k(a)}^k.$$

Algorithm 3 l -block insertion intra forward

```

1: procedure  $l$ -BLOCKINSERTIONINTRA( $\pi_k, f_{\pi_k}, l, C$ )
2:    $\pi_k^* \leftarrow \pi_k; f_k^* \leftarrow f_{\pi_k}$   $\triangleright \pi_k$  is the current sequence of machine  $k$  with associated cost  $f_{\pi_k}$  and  $\pi_k^*$  is the best improving neighbor sequence with associated cost  $f_k^*$ 
3:    $p_B \leftarrow 0$   $\triangleright p_B$  is the total processing time of the block considered for reinsertion
4:   for  $i' = 1, \dots, l$  do
5:      $p_B \leftarrow p_B + p_{\pi_k(i')}$ 
6:   for  $i = 1, \dots, n_k - l$  do  $\triangleright n_k$  is the number of jobs in machine  $k$ 
7:      $f_{\pi_k'} \leftarrow f_{\pi_k}$   $\triangleright f_{\pi_k'}$  is the cost of the neighbor sequence  $\pi_k'$  under evaluation
8:     for  $j = i + l, \dots, n_k - 1$  do
9:        $lateness \leftarrow C_{\pi_k(j)} - p_B - d_{\pi_k(j)}$ 
10:       $f_{\pi_k'} \leftarrow f_{\pi_k'} + \max\{w_{\pi_k(j)} \times lateness, w'_{\pi_k(j)} \times (-lateness)\}$ 
11:       $f_{\pi_k'} \leftarrow f_{\pi_k'} - f_{\pi_k(j)}$   $\triangleright f_{\pi_k(j)}$  is the cost of job  $\pi_k(j)$  in the sequence
12:       $f_{\pi_k'} \leftarrow f_{\pi_k'} + \text{CompCostBlock}(j, i, i + l - 1, \pi_k, \pi_k, C_{\pi_k(j)} - p_B)$ 
13:      if  $f_{\pi_k'} < f_k^*$  then
14:         $\pi_k^* \leftarrow \pi_k'; f_k^* \leftarrow f_{\pi_k'}$ 
15:       $p_B \leftarrow p_B - p_{\pi_k(i)}^k + p_{\pi_k(i+l)}^k$ 
16:   return  $f_k^*$ 

```

Nevertheless, since we adopted very small values for l and l' , as will be shown in Sect. 5, we decided to implement the move evaluation schemes with complexity $\mathcal{O}(l_{\max})$, where $l_{\max} = \max\{l, l'\}$, as also shown in Table 7 rather than $\mathcal{O}(1)$, which results in an overall complexity of $\mathcal{O}(l_{\max}n_k^2)$, but in practice it seems to offer a more interesting scalability, even for large size instances. This happens because the preprocessing phase requires an additional overhead for dealing with blocks of size $l_{\max} > 1$ that in the end does not compensate the advantage of performing the move evaluations in $\mathcal{O}(1)$ steps.

The preprocessing consists of precomputing the costs of the blocks that depend on g^k in a particular order. This enables the move evaluation procedure to access the cost of those blocks in constant time. For example, when performing the move evaluation of a neighbor solution after applying a move of the neighborhood l -block insertion intra, one can compute the associated cost in constant time by means of the following expression: $W_{i-1}^k + [g_i^k(t_2) - g_{i+l}^k(t_3)] + [g_{i+l}^k(t_1) - g_{j+1}^k(t_2)] + [W_{n_k}^k - W_j^k]$ (see Table 7). The terms of this expression that depend on W^k are precomputed in $\mathcal{O}(n_k)$ steps, as shown in (1), while the terms of this expression that depend on g^k can be precomputed in $\mathcal{O}(n_k^2)$ steps, as explained next.

For example, in order to store all values of $g_i^k(t_2 = C_j^k - \sum_{a=i}^{i+l-1} p_{\pi_k}^k(a))$ in $\mathcal{O}(n_k^2)$ steps, it is first necessary to sort the values of $C_j^k - \sum_{a=i}^{i+l-1} p_{\pi_k}^k(a)$ in ascending order. Note that this is possible to be achieved by sorting the values of C_j^k in ascending order, which can be done in $\mathcal{O}(n_k \log n_k)$ by using a standard sorting procedure. Next, for every i , it is now possible to compute the values of $g_i^k(t_2 = C_j^k - \sum_{a=i}^{i+l-1} p_{\pi_k}^k(a))$ in $\mathcal{O}(n_k)$ steps, because this function has

at most n segments, which leads to an overall complexity of $\mathcal{O}(n_k^2)$. A similar rationale can be applied to the other neighborhoods.

We will conclude this section by presenting alternative and simpler local search algorithms for the l -block insertion neighborhoods that do not rely on the auxiliary data structures and functions mentioned above. The overall complexity of the search using such algorithms is $\mathcal{O}(l_{\max}n_k^2)$ and they are also based on the work of Ergun and Orlin (2006).

In the case of the intra-machine neighborhood, two similar but distinct procedures are required to move a block forward and backward in the sequence, respectively. Algorithm 3 shows how the moves are evaluated when a block of size l is removed and reinserted in a forward position in the sequence. It can be observed that moves are evaluated in $\mathcal{O}(l)$ steps (lines 9–12) by making use of the move evaluation performed immediately before the current one. The same rationale can be applied for moving blocks of size l to a backward position in the sequence. Therefore, it can be verified that the overall complexity of Algorithm 3 is $\mathcal{O}(ln_k^2)$.

Algorithm 4 shows the pseudocode of the neighborhood l -block insertion inter-machine. Note that when a block of size l is removed from machine k , one needs to compute the cost of the new sequence of k only once and this can be done in $\mathcal{O}(n_k)$ steps as in Sect. 4.2.2. As for machine k' , where the block is going to be inserted in several positions, the cost of each modified sequence can be computed in $\mathcal{O}(l)$ using the same idea presented in Algorithm 3 after computing the cost of inserting the block in the first position in $\mathcal{O}(n_{k'})$ also as in Sect. 4.2.2. Hence, the overall complexity of Algorithm 4 is $\mathcal{O}(ln_k^2)$.

Algorithm 4 l -block insertion inter

```

1: procedure  $l$ -BLOCKINSERTIONINTER( $\pi, f, l$ )
2:    $\pi^* \leftarrow \pi, f^* \leftarrow f_\pi$   $\triangleright \pi$  is the current solution with associated cost  $f_\pi$  and  $\pi^*$  is the best improving neighbor with associated cost  $f^*$ 
3:   for  $k = 1, \dots, m$  do
4:     for  $k' = 1, \dots, m$  do
5:       if  $k \neq k'$  then
6:         for  $i = 1, \dots, n_k - l + 1$  do
7:            $f_{\pi'_k} \leftarrow W_{i-1}^k + \text{CompCostBlock}(i-1, i+l, n_k, \pi_k, \pi'_k, C_{\pi_k(i-1)})$   $\triangleright f_{\pi'_k}$  is the cost of the modified sequence  $\pi'_k$ 
8:            $\tilde{\pi}_{k'} \leftarrow \langle \pi_k(i), \dots, \pi_k(i+l-1) \rangle \oplus \langle \pi_{k'} \rangle$ 
9:            $f_{\pi'_{k'}} \leftarrow l\text{-blockInsertionIntraF}(\tilde{\pi}_{k'}, f_{\tilde{\pi}_{k'}}, l)$   $\triangleright f_{\pi'_{k'}}$  is the cost of the modified sequence  $\pi'_{k'}$ 
            $\pi'_{k'}$ 
10:          if  $f_\pi - f_{\pi_k} - f_{\pi_{k'}} + f_{\pi'_k} + f_{\pi'_{k'}} < f^*$  then
11:             $f^* \leftarrow f_\pi - f_{\pi_k} - f_{\pi_{k'}} + f_{\pi'_k} + f_{\pi'_{k'}}$ 
12:             $\pi^* \leftarrow \pi'$ 
13:   return  $f^*$ 

```

4.2.4 Move evaluation for problems with sequence-dependent setup times but without idle time

Liao et al. (2012) showed, for problem $1|s_{ij}|w_jT_j$, that is possible to perform the move evaluations of the same neighborhoods considered by Ergun and Orlin (2006) in amortized $\mathcal{O}(1)$ time, but at the expense of preprocessing the required auxiliary data structures in $\mathcal{O}(n^2 \log n)$ operations. Their method is based on the one of the latter authors and it can be extended to solve problem $R|s_{ij}^k|\sum w_j^kE_j + w_jT_j$ and its particular cases (only those without idle time) by using the same rationale presented in Sect. 4.2.3.

The main difference between the approaches of Liao et al. (2012) and Ergun and Orlin (2006) is the preprocessing phase, where the sorting procedure should be called for each i before computing the values of $g_j^k(t)$ for each j in the sequence, because t now depends on the existing setup times, thus leading to a complexity of $\mathcal{O}(n^2 \log n)$. This additional step is necessary for properly updating the functions $g_j^k(t)$ in order to perform the move evaluation in $\mathcal{O}(1)$ time (or alternatively in $\mathcal{O}(l)/\mathcal{O}(\max\{l, l'\})$ time) using the expressions presented in Table 7.

Nevertheless, this approach seems to be indicated for large scale instances, as suggested by the experiments performed in Liao et al. (2012). For small-medium scale instances the procedure described in Sect. 4.2.2 appears to be more suitable to address the problems with sequence-dependent setup times listed in Table 1.

4.2.5 Move evaluation for problems with idle time

Ibaraki et al. (2005) proposed an efficient approach for evaluating the moves during the local search when idle times are considered. In this case, the *timing problem*, which consists of determining the optimal start time of each job in the sequence, should also be solved when evaluating the cost of a solution. The authors first present a dynamic programming-

based algorithm to solve this problem and then they show how to integrate such method within the local search.

As in Ergun and Orlin (2006), the method of Ibaraki et al. (2005) relies on auxiliary data structures and functions. The idea is to determine functions that return the minimum penalty of the subsequences (blocks) when starting at a particular time t , both forward and backward. The cost of the solution under evaluation is thus computed by using such information over the resulting sequence obtained after concatenating certain subsequences.

In Ibaraki et al. (2008) the authors suggested a balanced binary tree-based implementation that allows for solving the timing problem in $\mathcal{O}(n_k \log \delta_k)$ time, where δ_k denotes the total number of segments of the penalty function for the jobs sequenced in machine k . As demonstrated by the authors, a move can be evaluated in amortized $\mathcal{O}(\log \delta_{\max})$ time, where $\delta_{\max}$ is the largest δ_k among the machines involved. The necessary functions of each machine k can be precomputed in $\mathcal{O}(\delta_k \log \delta_k)$ steps. However, for simplicity, we used a linked list-based implementation (Ibaraki et al. 2005), which yields an amortized complexity of $\mathcal{O}(\delta_{\max})$ where the necessary functions for each machine k can be precomputed in $\mathcal{O}(n_k \delta_k)$ steps.

For the neighborhoods considered in this work, the modified solutions can be obtained by concatenating at most three subsequences, as described in detail in Ibaraki et al. (2005, 2008).

4.3 Perturbation mechanisms

The following three perturbation mechanisms were implemented:

- (l, l') -block swap intra-machine: one (l, l') -block swap intra-machine move is performed at random with l and $l' \in \{2, \dots, n/4\}$.
- Multiple $(1, 1)$ -insertion inter-machine: a job j from a machine k is moved to a machine k' , while a job j' from

Table 8 Selected problems with publicly available instances

Problem	#Instances	$ J $	$ M $	Related work
$1 \sum w_j T_j$	875 ^a	40 to 300	1	Tanaka et al. (2009)
$1 \sum w'_j E_j + w_j T_j$	875 ^a	40 to 300	1	Tanaka et al. (2009)
$1 \sum w'_j E_j + w_j T_j$	5276 ^b	20 to 150	1	Sourd and Kedad-Sidhoum (2008), Sourd (2009), Tanaka and Fujikuma (2012)
$1 r_j \sum w'_j E_j + w_j T_j$	2400 ^b	20 to 200	1	Bülbül et al. (2007), Tanaka and Fujikuma (2012)
$1 r_j \sum w_j C_j$	1900 ^c	20 to 200	1	Pan and Shi (2008), Tanaka and Fujikuma (2012)
$P \sum T_j$	2250 ^d	20 and 25	2 to 10	Tanaka and Araki (2008)
$P \sum w_j T_j$	1125 ^e	40 to 100	2, 4, 10, 20 and 30	Rodrigues et al. (2008), Pessoa et al. (2010)
$P \sum w'_j E_j + w_j T_j$	5000 ^f	40 to 500	2, 4, 10, 20 and 30	Amorim et al. (2013), Amorim (2013)
$R \sum w_j T_j$	1440 ^g	40 to 200	2 to 5	Şen and Bülbül (2015)
$R \sum w'_j E_j + w_j T_j$	1440 ^g	40 to 200	2 to 5	Şen and Bülbül (2015)

^a Available at: <https://sites.google.com/site/shunjitanaka/sips>

^b Available at: <http://www-poleia.lip6.fr/~safia/et/>

^c Available at: <http://pages.cs.wisc.edu/~yunpeng/test/sm/dwct/instances.htm>

^d Available at: <https://sites.google.com/site/shunjitanaka/pmtt>

^e Available at: <http://algorithms.icomp.ufam.edu.br/index.php/benchmark-instances/weighted-tardiness-scheduling>

^f Available at: <http://algorithms.icomp.ufam.edu.br/index.php/benchmark-instances/weighted-earliness-tardiness-scheduling>

^g Available at: http://people.sabanciuniv.edu/~bulbul/papers/Sen_Bulbul_Rm_TWT-TWET_source-data-results_IJOC_2015.rar

k' is moved to k . The jobs, machines and positions to be inserted are all chosen at random and such procedure is repeated one, two or three consecutive times.

- Multiple (l, l') -block insertion inter-machine: this perturbation generalizes the previous one but in this case blocks of jobs of sizes l and l' , respectively, are involved in the move with $l \in \{1, 2\}$ and $l' \in \{2, 3\}$.

The first mechanism is only applied for single machine problems, while the second and third are only applied for parallel machine problems and they are chosen at random.

5 Computational experiments

The UILS algorithm was coded in C++ and the experiments were executed in an Intel Core i7-2600 with 3.40 GHz and 16 GB of RAM running under Ubuntu Linux 12.04. Only a single thread was used in our testing. The proposed algorithm was executed 10 times for each instance in the final experiments. Furthermore, with a view of better comparing the runtime performance between UILS and other methods from the literature that used machines with a quite inferior hardware performance than ours, we have scaled the CPU time values reported in their work to our machine. This was done by means of approximation factors that were computed using the single thread rating values reported in <https://www.cpubenchmark.net/>.

Unfortunately, the vast majority of the problems listed in Table 1 does not have publicly available instances. Most authors usually generate the instances for a particular problem themselves but they seldom make them easily accessible for the research community. Table 8 lists some problems in which we are aware that there are instances available online. We executed the UILS algorithm only for those with results reported in the literature.

We remark that we do not present the results obtained by our algorithm for the publicly available instances of problems $1|s_{ij}| \sum T_j$ and $1|s_{ij}| \sum w_j T_j$, because those found by a similar and simplified version of this algorithm were already reported in Subramanian et al. (2014). UILS still finds slightly better results than the simplified algorithm, but for brevity we chose not report them here.

5.1 Parameter tuning

In this section we explain how we tuned the main parameters of UILS, that is, I_R , I_{ILS} , and the neighborhood sets L_{intra} , L'_{intra} , L_{inter} and L'_{inter} .

We start by describing how we selected the neighborhood sets. We used an incremental approach where we included one neighborhood at a time in ascending order of l and l' , and then we evaluated the impact on the solution quality after such inclusion. More precisely, for every new neighborhood considered, we applied the RVND procedure over an initial solution and then we computed the value of the gap of the

Table 9 Improvements found by RVND when including a new neighborhood

Neighborhoods	Avg. imp. (%)
1-block insertion intra-machine	64.22
+ 2-block insertion intra-machine	64.33
+ 3-block insertion intra-machine	63.33
+ (1.1)-swap intra-machine	65.75
+ (1.2)-swap intra-machine	65.63
+ 1-block insertion inter-machine	67.50
+ 2-block insertion inter-machine	68.01
+ 3-block insertion inter	67.62
+ (1.1)-swap inter-machine	68.04
+ (1.2)-swap inter-machine	68.32
+ (1.3)-swap inter-machine	69.02
+ (2.2)-swap inter-machine	69.22
+ (2.3)-swap inter-machine	69.47
+ (2.4)-swap inter-machine	69.58
+ (3.3)-swap inter-machine	69.63
+ (3.4)-swap inter-machine	69.65
+ (4.4)-swap inter-machine	69.79
+ (4.5)-swap inter-machine	69.63

resulting local optimum with respect to the initial solution. Initially, only one neighborhood of a particular type was considered, namely 1-block insertion intra-machine, and then we kept increasing the set until no improvement was verified. In this case, we stopped adding the l -block insertion intra-machine neighborhoods and started including (l, l') -block swap intra-machine neighborhoods until no improvement was observed. Then we included the l -block insertion inter-machine neighborhoods and, finally, the (l, l') -block swap intra-machine neighborhoods.

We selected 24 challenging instances among those available for problem $P|| \sum w'_j E_j + w_j T_j$ (without idle time) to determine the neighborhood sets. For each instance and for each combination of neighborhoods, we ran the RVND procedure 5 times over different initial solutions and we stored the average improvement obtained. Table 9 reports the average improvements obtained for each combination. Strikethrough entries correspond to those neighborhoods that have been disregarded due to lack of average improvement on the solution quality. The following sets were selected: $L_{\text{intra}} = \{1, 2\}$, $L'_{\text{intra}} = \{(1, 1)\}$, $L_{\text{inter}} = \{1, 2\}$ and $L'_{\text{inter}} = \{(1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (2, 4), (3, 3), (3, 4), (4, 4)\}$.

It is worth mentioning that for single machine problems we considered $L_{\text{intra}} = \{1, 2, 3\}$ and $L'_{\text{intra}} = (1, 1)$ as in Subramanian et al. (2014).

In order to tune the parameter I_{ILS} we performed a series of experiments for the same 24 instances previously selected.

We considered different values, namely, $n/2$, n , $2 \times n$, $4 \times n$ and $6 \times n$. We noticed that a single start of the method, i.e., $I_R = 1$, led to inconclusive results. Hence, we decided to adopt $I_R = 10$ as in Subramanian et al. (2014) in order to better calibrate the value of I_{ILS} . Table 10 shows the results obtained with different values of I_{ILS} where the gap reported for every instance is between the average solution of 5 runs and the best known solution (BKS). We also report the average time of the 5 runs, as well as the number of times BKS was found or improved.

The results illustrated in Table 10 suggest that $4 \times n$ and $6 \times n$ seem to be superior in terms of solution quality than the other settings, but not very different from each other. We decided to adopt $I_{\text{UILS}} = 4 \times n$ because it seemed to provide an interesting compromise between solution quality and average CPU time.

We performed similar experiments considering $I_R = 15$ and $I_R = 20$, but the increment in the CPU time due to more restarts do not seem to compensate the slight increase in the solution quality. Therefore, we chose to keep $I_R = 10$, but limiting the total execution time to 600 seconds in order to avoid long runs for large instances.

While performing further experiments, we realized that the proposed algorithm performed quite slow for problems with idle time when adopting $I_{\text{ILS}} = 4 \times n$. This is because the local search phase is more expensive in terms of CPU time, as already explained. We thus performed a similar experiment as the one just described above, but for 10 selected instances of problem $R|| \sum w'_j E_j + w_j T_j$. In this case we only considered the following values for I_{ILS} : $n/2$, n and $2 \times n$. Furthermore, since UILS systematically found or improved the BKS, we decided to evaluate the gap with respect to the lower bound and also with respect to our best solution (OBS) so as to better appreciate the influence of the parameter. The results of these experiments are reported in Table 11. By following a similar criterion used in the previous case, we decided to adopt $I_{\text{ILS}} = n$ for problems with idle time.

5.2 Results for single machine (E-)T problems

This section presents the results obtained by the UILS algorithm for some single machine (E-)T problems, more precisely, $1|| \sum w_j T_j$, $1|| \sum w'_j E_j + w_j T_j$ (without idle time), $1|| \sum w'_j E_j + w_j T_j$ (with idle time), $1|r_j| \sum w'_j E_j + w_j T_j$ and $1|r_j| \sum w_j C_j$. As already mentioned, the highly sophisticated exact algorithms presented in Tanaka et al. (2009), Tanaka and Fujikuma (2012) are the best ones available for such problems. Since all optimal solutions are known for the instances considered in this section, we decided to stop the algorithm if: (i) the optimal solution is found; or (ii) the CPU time spent by UILS exceeds a predefined time limit. We have imposed a limit of 300 seconds for all problems except

Table 10 Results for different values of I_{ILS} in selected instances of problem $P || \sum w'_j E_j + w_j T_j$ (without idle time)

Instance	$I_{ILS} = n/2$			$I_{ILS} = n$			$I_{ILS} = 2 \times n$			$I_{ILS} = 4 \times n$			$I_{ILS} = 6 \times n$		
	Gap (%)	Time (s)	#BKS	Gap (%)	Time (s)	#BKS	Gap (%)	Time (s)	#BKS	Gap (%)	Time (s)	#BKS	Gap (%)	Time (s)	#BKS
wet100-10m-121	0.08	20.5	0	0.06	31.9	0	0.05	57.2	0	0.03	111.0	0	0.03	172.0	0
wet100-10m-31	0.78	29.2	0	0.63	52.7	0	0.47	96.6	0	0.37	173.5	0	0.29	264.6	0
wet100-10m-61	0.43	24.5	0	0.34	39.1	0	0.25	76.6	0	0.18	116.5	0	0.09	182.5	1
wet100-2m-1	0.02	34.2	0	0.01	64.2	0	0.01	148.9	0	0.00	262.4	0	0.00	411.1	1
wet100-2m-11	0.04	30.6	1	0.02	67.5	0	0.02	101.1	0	0.00	174.8	3	0.01	244.5	2
wet100-2m-111	0.08	34.0	0	0.05	67.6	0	0.00	136.5	2	0.01	219.3	2	0.00	307.8	1
wet100-2m-121	0.01	20.5	0	0.01	40.1	0	0.00	83.2	1	0.00	131.5	2	0.00	209.4	4
wet100-2m-31	0.07	34.0	0	0.02	64.0	0	0.02	121.4	0	0.00	227.0	1	0.00	347.4	2
wet100-2m-61	0.13	31.3	0	0.04	55.8	0	0.02	95.4	0	0.00	174.3	5	0.00	234.3	3
wet100-2m-71	0.00	17.0	0	0.00	32.7	1	0.00	58.5	2	0.00	99.7	2	0.00	140.7	3
wet100-2m-81	0.37	37.1	0	0.25	65.4	0	0.17	109.1	0	0.10	244.9	1	0.08	293.7	1
wet100-2m-91	0.02	25.4	1	0.01	47.8	2	0.00	81.4	4	0.00	125.8	4	0.00	182.0	5
wet100-4m-111	0.29	35.1	0	0.26	80.3	0	0.20	131.1	0	0.09	238.5	0	0.14	337.6	0
wet100-4m-31	0.44	33.8	0	0.23	72.6	0	0.26	134.5	0	0.19	274.1	0	0.17	374.5	0
wet100-4m-61	0.30	33.5	0	0.22	66.2	0	0.18	102.9	0	0.15	219.0	0	0.06	271.6	1
wet100-4m-81	2.06	38.6	0	1.89	75.9	0	1.56	143.8	0	0.91	248.7	0	0.85	379.6	0
wet150-2m-1	0.02	124.0	0	0.02	257.8	0	0.01	564.7	0	0.01	1044.1	0	0.00	1569.2	0
wet200-2m-1	0.02	385.9	0	0.02	825.3	0	0.01	1725.8	0	0.01	3603.8	0	0.01	5201.9	0
wet40-2m-1	0.02	1.4	0	0.00	2.5	3	0.00	5.8	5	0.00	9.1	5	0.00	11.8	5
wet40-4m-1	0.24	1.5	0	0.10	3.1	1	0.02	5.6	3	0.02	11.0	3	0.00	14.2	5
wet40-4m-111	0.01	1.3	4	0.00	2.8	5	0.00	4.3	5	0.00	7.6	5	0.00	11.4	5
wet40-4m-121	0.00	0.9	5	0.00	1.6	5	0.00	2.6	5	0.00	4.5	5	0.00	6.2	5
wet40-4m-91	0.03	1.1	4	0.00	1.8	5	0.00	3.4	5	0.00	6.7	5	0.00	9.3	5
wet50-2m-1	0.02	3.2	0	0.02	5.4	0	0.01	10.8	0	0.00	19.4	1	0.00	28.9	0
Avg.	0.23	41.6	—	0.18	84.3	—	0.14	166.7	—	0.09	322.8	—	0.07	466.9	—
Total	—	—	15	—	—	22	—	—	32	—	—	44	—	—	49

Table 11 Results for different values of I_{ILS} in selected instances of problem $R||\sum w'_j E_j + w_j T_j$ (with idle time)

Instance	$I_{ILS} = n/2$				$I_{ILS} = n$				$I_{ILS} = 2 \times n$			
	Gap _{LB} (%)	Gap _{Obs} (%)	Time (s)	#OBS	Gap _{LB} (%)	Gap _{Obs} (%)	Time (s)	#OBS	Gap _{LB} (%)	Gap _{Obs} (%)	Time (s)	#OBS
WET40_R2_25_100_021	1.87	0.01	18.3	4	1.86	0.00	31.7	5	1.86	0.00	59.5	5
WET40_R2_25_100_101	0.00	0.00	5.8	5	0.00	0.00	10.2	5	0.00	0.00	18.7	5
WET60_R2_25_100_021	1.25	0.07	131.2	1	1.18	0.00	245.3	5	1.19	0.02	391.3	4
WET60_R2_25_100_101	0.08	0.00	33.3	5	0.08	0.00	60.4	5	0.08	0.00	102.9	5
WET60_R3_25_100_021	0.05	0.04	102.7	3	0.01	0.00	204.2	4	0.01	0.00	363.1	5
WET60_R3_25_100_101	0.01	0.00	31.6	5	0.01	0.00	53.5	5	0.01	0.00	95.6	5
WET80_R2_25_100_021	1.24	0.13	547.5	1	1.14	0.03	951.4	1	1.12	0.01	1534.3	3
WET80_R2_25_100_101	0.02	0.00	119.5	5	0.02	0.00	233.1	5	0.02	0.00	390.6	5
WET90_R3_25_100_021	1.43	0.39	656.8	0	1.09	0.05	1307.7	1	1.09	0.05	2010.7	2
WET90_R3_25_100_101	0.16	0.00	162.4	5	0.16	0.00	253.3	5	0.16	0.00	409.0	5
Avg.	0.61	0.06	180.9	—	0.56	0.01	335.1	—	0.55	0.01	537.6	—
Total	—	—	—	34	—	—	—	41	—	—	—	44

Table 12 Summary of results for problem $1||\sum w_j T_j$

Instance group	#Inst	SiPS Time ^a (s)	UILS				
			Gap _{best} (%)	Gap _{avg} (%)	Time _{avg} (s)	#Best	#Opt
wt040	125	< 0.1	0.00	0.00	< 0.1	1250	125
wt050	125	0.1	0.00	0.00	< 0.1	1250	125
wt100	125	1.0	0.00	0.00	0.2	1250	125
wt150	125	3.9	0.00	0.00	1.3	1250	125
wt200	125	11.1	0.00	0.00	10.1	1250	125
wt250	125	25.1	0.00	0.00	19.9	1250	125
wt300	125	49.7	0.00	0.00	37.6	1250	125
Total	875	–	–	–	–	8750	875
Avg.	–	13.0	0.0	0.0	9.9	–	–

^aIntel Pentium 4 with 3.4 GHz scaled to our Intel i7 with 3.40 GHz**Table 13** Summary of results for problem $1||\sum w'_j E_j + w_j T_j$ —without idle time

Instance group	#Inst	SiPS Time ^a (s)	UILS				
			Gap _{best} (%)	Gap _{avg} (%)	Time _{avg} (s)	#Best	#Opt
wet040	125	< 0.1	0.00	0.00	0.1	1250	125
wet050	125	0.1	0.00	0.00	0.3	1250	125
wet100	125	1.4	0.00	0.00	4.4	1250	125
wet150	125	6.9	0.00	0.00	23.1	1250	125
wet200	125	21.7	0.00	0.00	75.9	1250	125
wet250	125	53.4	0.00	0.00	165.3	1196	125
wet300	125	117.5	0.00	0.01	252.2	899	120
Total	875	–	–	–	–	8345	870
Avg.	–	28.7	0.00	0.00	74.5	–	–

^aIntel Pentium 4 with 3.4 GHz scaled to our Intel i7 with 3.40 GHz

$1|r_j|\sum w'_j E_j + w_j T_j$, where a limit of 3600 seconds was adopted. In addition, we have also already mentioned that we are not aware of heuristic algorithms whose performance are comparable to those of Tanaka et al. (2009), Tanaka and Fujikuma (2012), especially when idle times are considered, meaning that the CPU time spent by current heuristic methods to find high-quality (or even optimal) solutions tend to be much higher than those of the referred exact methods. Therefore, in order to prevent long executions, we have modified the stopping criterion when conducting the experiments for these instances.

We ran UILS 10 times for each instance and we report the best (Gap_{best}) and average (Gap_{avg}) gaps with respect to the optimal, the average CPU time in seconds (Time_{avg} (s)), and the number of BKSs (#Best) and optimal solutions (#Opt) obtained.

Tables 12 and 13 present the results for problems $1||\sum w_j T_j$ and $1||\sum w'_j E_j + w_j T_j$ (without idle time), respectively. A comparison is performed with those obtained by the exact method of Tanaka et al. (2009). We can observe that UILS is capable of finding all known 1245 optimal solutions in a comparable CPU time.

Table 14 shows the results found by UILS for the set of instances suggested by Sourd and Kedad-Sidhoum (2008) for problem $1||\sum w'_j E_j + w_j T_j$ (with idle time). We compare our results with those achieved by the exact method of Tanaka and Fujikuma (2012). The proposed algorithm managed to find all known optimal solutions, except for 2 out of 5276 instances. However, as already discussed, because of the existence of idle times, the runtime performance of UILS does not tend to be as competitive as those of the referred exact algorithm.

Tables 15 and 16 illustrate the results found by UILS for problems involving release dates, in particular, $1|r_j|\sum w'_j E_j + w_j T_j$ and $1|r_j|\sum w_j C_j$. A comparison is performed with the results obtained by the exact algorithm of Tanaka and Fujikuma (2012). We can verify that the inclusion of release dates, which naturally induce solutions with idle times, does not seem to substantially affect the overall performance of the exact algorithm, as opposed to UILS, which is visibly more sensitive to the presence of release dates. Yet, the average gaps obtained for each group of instances of problems $1|r_j|\sum w'_j E_j + w_j T_j$ and $1|r_j|\sum w_j C_j$ were always less than or equal to 0.78 and 0.08%, respectively.

Table 14 Summary of results for problem $1||\sum w'_j E_j + w_j T_j$ —with idle time

Instance group	#Inst	SiPSi Time ^a (s)	UILS				
			Gap _{best} (%)	Gap _{avg} (%)	Time _{avg} (s)	#Best	#Opt
sks20	1274	< 0.1	0.00	0.00	0.3	12740	1274
sks30	1274	0.1	0.00	0.00	2.0	12740	1274
sks40	1274	0.3	0.00	0.00	8.6	12740	1274
sks50	1274	0.6	0.00	0.00	28.7	12740	1274
sks60	45	1.3	0.00	0.00	68.8	450	45
sks90	45	6.1	0.00	0.16	214.6	245	44
sks120	45	11.9	0.00	0.14	128.6	404	45
sks150	45	22.5	0.00	0.28	284.2	357	44
Total	5276	—	—	—	—	52416	5274
Avg.	—	0.6	0.00	0.07	15.5	—	—

^aIntel Pentium 4 with 3.4 GHz scaled to our Intel i7 with 3.40 GHz

Table 15 Summary of results for problem $1|r_j|\sum w'_j E_j + w_j T_j$

Instance group	#Inst	SiPSi Time ^a (s)	UILS				
			Gap _{best} (%)	Gap _{avg} (%)	Time _{avg} (s)	#Best	#Opt
bky20	300	< 0.1	0.00	0.00	0.2	3000	300
bky40	300	0.1	0.00	0.00	17.6	2999	300
bky60	300	0.4	0.01	0.08	186.6	1991	277
bky80	300	1.3	0.08	0.34	282.1	556	143
bky100	300	6.8	0.28	0.78	306.9	89	37
bky130 ^b	300	18.7	0.05	0.20	3475.6	239	87
bky170 ^b	300	57.2	0.21	0.47	3598.6	5	2
bky200 ^b	300	110.8	0.34	0.66	3600.0	4	3
Total	2400	—	—	—	—	8883	1149
Avg.	—	24.4	0.12	0.31	1433.5	—	—

^aIntel Pentium 4 with 3.4 GHz scaled to our Intel i7 with 3.40 GHz.

^b Time limit of 1 h

5.3 Results for problem $P||\sum T_j$

The optimal solutions of all 2250 instances available for problem $P||\sum T_j$ were found by the exact method of Tanaka and Araki (2008). The UILS algorithm was capable of finding such optima in at least 9 of the 10 runs and the average computational time was smaller than one second, as can be observed in Table 17.

5.4 Results for problem $P||\sum w_j T_j$

Tables 18 and 19 present the aggregate results found for the instances considered in Rodrigues et al. (2008) and Pessoa et al. (2010), respectively. Each group contains 25 instances selected by the authors and has the format wtX–Ym, where X denotes the number of jobs and Y corresponds to the number of machines of the group. For example, wt40–2m indicates that the group is composed of instances containing 40 jobs and 2 machines. Detailed results are found in Tables 27–32 in Appendix A.1 of ESM. The optimal solutions of all

instances of groups wt40–2m, wt40–4m, wt50–2m and wt50–4m were reported in Rodrigues et al. (2008). The gaps shown in Table 18 are with respect to such optimal solutions, whereas those presented in Table 19 are with respect to the best known solutions. This is because some instances of groups wt100–2m, wt100–4m are still open. Pessoa et al. (2010) reported the best upper bounds for those cases where the optimal solutions were not found by their exact method.

From the results obtained, it can be observed that UILS clearly outperforms the heuristic of Rodrigues et al. (2008) for the instances containing 40 and 50 jobs, respectively. High-quality solutions were also obtained for 100-job instances and 7 improved solutions were found.

5.5 Results for problem $P||\sum w'_j E_j + w_j T_j$

Table 20 shows a summary of the results found for problem $P||\sum w'_j E_j + w_j T_j$, without idle time. Each group of instance has the format wetX–Ym, where X indicates the number of jobs and Y denotes the number of machines of the

Table 16 Summary of results for problem $1|r_j|\sum w_j C_j$

Instance group	#Inst	SiPSi Time ^a (s)	UILS				
			Gap _{best} (%)	Gap _{avg} (%)	Time _{avg} (s)	#Best	#Opt
PanShi20	100	0.0	0.00	0.00	0.0	1000	100
PanShi30	100	0.1	0.00	0.00	0.0	1000	100
PanShi40	100	0.2	0.00	0.00	0.1	1000	100
PanShi50	100	0.4	0.00	0.00	0.8	1000	100
PanShi60	100	0.9	0.00	0.00	1.6	1000	100
PanShi70	100	1.6	0.00	0.00	6.5	992	100
PanShi80	100	2.6	0.00	0.00	23.7	963	98
PanShi90	100	4.2	0.00	0.00	31.9	958	98
PanShi100	100	6.2	0.00	0.00	75.8	834	92
PanShi110	100	8.8	0.00	0.01	87.7	795	94
PanShi120	100	12.1	0.00	0.01	109.9	700	86
PanShi130	100	17.5	0.00	0.02	124.6	671	83
PanShi140	100	21.1	0.01	0.03	132.0	628	74
PanShi150	100	30.3	0.01	0.03	139.3	603	68
PanShi160	100	38.3	0.02	0.04	149.8	581	67
PanShi170	100	46.8	0.02	0.04	157.4	549	66
PanShi180	100	53.1	0.02	0.05	170.5	521	59
PanShi190	100	74.6	0.03	0.06	163.2	534	57
PanShi200	100	96.6	0.04	0.08	178.9	510	57
Total	1900	—	—	—	—	14,839	1599
Avg.	—	21.9	0.01	0.02	81.8	—	—

^aIntel Pentium 4 with 3.4 GHz scaled to our Intel i7 with 3.40 GHz**Table 17** Summary of the results for problem $P||\sum T_j$

Instance group	#Inst	Tanaka and Araki (2008) Time ^a (s)	UILS				
			Gap _{best} (%)	Gap _{avg} (%)	Time _{avg} (s)	#Best	#Opt
N20_M2	125	0.2	0.00	0.00	0.3	1250	125
N20_M3	125	0.1	0.00	0.00	0.3	1250	125
N20_M4	125	0.1	0.00	0.00	0.3	1250	125
N20_M5	125	< 0.1	0.00	0.00	0.3	1250	125
N20_M6	125	< 0.1	0.00	0.00	0.3	1250	125
N20_M7	125	< 0.1	0.00	0.00	0.3	1250	125
N20_M8	125	< 0.1	0.00	0.00	0.3	1244	125
N20_M9	125	< 0.1	0.00	0.00	0.2	1247	125
N20_M10	125	< 0.1	0.00	0.00	0.2	1248	125
N25_M2	125	0.5	0.00	0.00	0.6	1250	125
N25_M3	125	6.3	0.00	0.00	0.6	1247	125
N25_M4	125	15.5	0.00	0.00	0.6	1248	125
N25_M5	125	4.7	0.00	0.00	0.6	1244	125
N25_M6	125	0.1	0.00	0.00	0.6	1246	125
N25_M7	125	0.1	0.00	0.00	0.5	1250	125
N25_M8	125	< 0.1	0.00	0.00	0.5	1248	125
N25_M9	125	0.1	0.00	0.00	0.5	1243	125
N25_M10	125	< 0.1	0.00	0.00	0.4	1250	125
Total	2250	—	—	—	—	22,465	2250
Avg.	—	1.5	0.00	0.00	0.4	—	—

^aIntel Pentium 4 with 2.4 GHz scaled to our Intel i7 with 3.40 GHz

Table 18 Summary of results for problem $P||\sum w_j T_j$

Instance group	Rodrigues et al. (2008)				UILS				
	Best run		Average		Best run			Average	
	Gap (%)	#BKS	Gap (%)	Time ^a (s)	Gap (%)	#Equal to BKS	#Imp	Gap (%)	Time (s)
wt40-2m	0.00	24	0.87	6.0	0.00	25	0	0.01	3.1
wt40-4m	0.00	25	3.29	20.0	0.00	25	0	0.00	4.0
wt50-2m	0.00	25	1.85	14.8	0.00	25	0	0.00	6.5
wt50-4m	0.00	25	3.08	50.6	0.00	25	0	0.00	8.2
Total	–	99	–	–	–	100	0	–	–
Avg.	0.00	–	2.27	22.8	0.00	–	–	0.00	5.5

^a Average of $30 \times m \times n$ runs in an Intel Xeon 2.33 GHz

Table 19 Summary of results for problem $P||\sum w_j T_j$

Instance group	Pessoa et al. (2010)		UILS				
	Best run		Best run			Average	
	Gap (%)	#BKS	Gap (%)	#Equal to BKS	#Imp	Gap (%)	Time (s)
wt100-2m	0.00	21	0.00	22	2	0.00	71.1
wt100-4m	0.00	16	−0.01	15	5	0.00	94.8
Total	–	37	–	37	7	–	–
Avg.	0.00	–	0.00	–	–	0.00	83.0

group. The 2-machine and 4-machine groups have each of them 11 or 12 instances with available results for comparison, while the groups with 10 machines have 5 instances each. We compare our results with the best heuristics proposed in Amorim et al. (2013), Amorim (2013), that is, ILS+PR, GA+LS+PR and ILS-M. Detailed results are provided in Tables 33–41 in Appendix A.2 of ESM. Note that for some instances our best solution has one machine less.

For the instances of groups wt40-2m and wt50-2m, UILS found or improved all best known solutions. The same happened for group wt100-2m, except for a single instance where our best solution was only one unit above the BKS. For the 4-machine instances, our algorithm found or improved the BKS for all but five instances from the literature. Moreover, UILS improved one solution and equaled the results of the other ones for the instances of groups wt40-10m and wt50-10m. Finally, for the instances of group wt100-10m, UILS improved just one solution, but the average gap was only 0.24%. As for the CPU time, it can be observed that UILS is considerably faster than the other algorithms and the machines used have comparable configurations.

We also performed further experiments for the same set of instances, but now allowing idle times to be inserted, thus enabling us to evaluate the performance of the algorithm when considering this scenario. In Table 21, we compare the results obtained by allowing idle times with the BKSs found without idle times (see Tables 33–41 of ESM). As expected, better solutions were achieved by allowing idle times, but

at the expense of CPU time. More specifically, the average improvement was 7.4%, but UILS performed, on average, roughly 4 times slower. The table also shows that there was an improvement on the solution quality of 35 instances, while the results of another 32 were equaled. Detailed results on the 85 instances are presented in Tables 42–50 in Appendix A.2 of ESM.

5.6 Results for problem $R||\sum w_j T_j$

There are 12 groups of instances for problem $R||\sum w_j T_j$, each of them containing 60 test-problems, with format Nx_xMy, where _{xx} indicates the number of jobs and _y the number of machines. We compare our aggregate results with those achieved by Şen and Bülbül (2015) in Table 22. For each instance group, we report the number of optimal solutions (#Opt), improvements (#Imp), and BKSs found (#Equal to BKS), as well as the gaps with respect to the best lower bounds reported by Şen and Bülbül (2015) (Gap_{LB}), which were obtained either by a preemption-based relaxation that was solved using Benders' decomposition or CPLEX, or by the lower bound obtained through a time-indexed formulation that was solved using CPLEX. The upper bounds were found by applying the single machine solver SiPS/SiPSi (Tanaka et al. 2009; Tanaka and Fujikuma 2012) over each machine after running the Benders' decomposition. Finally, we also report the total average CPU time (Time (s)) spent by both algorithms, and the average CPU time required by UILS to find or improve

Table 20 Summary of results for problem $P || \sum w'_j E_j + w_j T_j$ —without idle time

Instance Group	ILS+PR				GA+LS+PR				ILS-M				UIIS				
	Best run		Average		Best run		Average		Best run		Average		Best run		Average		
	Gap (%)	#BKS	Gap (%)	Time ^a (s)	Gap (%)	#BKS	Gap (%)	Time ^a (s)	Gap (%)	#BKS	Gap (%)	Time ^a (s)	Gap (%)	#Equal to BKS	#Imp	Gap (%)	Time (s)
wet40-2m	0.00	12	0.00	15.4	0.00	12	0.01	33.6	0.00	12	0.00	13.6	−0.58	11	1	−0.58	5.6
wet50-2m	0.00	12	0.00	41.2	0.00	12	0.02	99.1	0.00	12	0.00	32.2	−0.78	11	1	−0.78	12.6
wet100-2m	0.02	5	0.02	588.0	0.00	10	0.01	3098.2	0.00	11	0.00	533.0	−0.67	10	1	−0.66	168.5
wet40-4m	0.74	11	0.74	68.4	0.74	11	0.75	201.1	0.74	11	0.74	37.0	0.00	12	0	0.00	6.3
wet50-4m	0.00	10	0.00	163.1	0.02	8	0.02	546.8	0.00	10	0.00	73.5	−0.91	10	1	−0.63	14.1
wet100-4m	0.14	2	0.16	3047.1	0.01	4	0.01	22,689.9	0.01	8	0.03	1876.4	0.08	5	1	0.15	190.3
wet40-10m	0.00	5	0.00	325.6	0.00	5	0.00	4780.4	0.00	5	0.00	46.9	0.00	5	0	0.00	4.1
wet50-10m	0.03	4	0.03	852.7	0.00	5	0.01	10,237.8	0.00	5	0.01	130.0	−0.60	4	1	−0.59	9.3
wet100-10m	1.43	0	1.45	16817.9	1.19	1	1.19	132,735.9	1.19	0	1.23	8424.8	−0.01	0	1	0.05	140.4
Avg.	0.26	—	0.27	2435.5	0.22	—	0.22	19,380.3	0.22	—	0.22	1240.8	−0.39	—	—	−0.34	61.3
Total	—	61	—	—	—	68	—	—	—	74	—	—	—	68	7	—	—

^a Average of 3 runs in an Intel i7-3770 3.40 GHz with 12 GB of RAM

Table 21 Summary of results for problem $P|| \sum w'_j E_j + w_j T_j$ —with idle time

Instance Group	Best run			Average	
	Gap ^a (%)	#Equal ^a to BKS	#Imp ^a	Gap ^a (%)	Time (s)
wet40-2m	−8.44	9	3	−8.43	30.5
wet50-2m	−10.15	6	5	−10.10	71.3
wet100-2m	−12.20	2	4	−12.10	600.0
wet40-4m	−6.23	6	6	−6.21	20.1
wet50-4m	−3.97	6	4	−3.92	53.1
wet100-4m	−5.82	0	3	−5.63	590.4
wet40-10m	−3.63	2	3	−3.63	7.2
wet50-10m	−5.76	1	4	−5.76	19.6
wet100-10m	−7.04	0	3	−6.89	480.1
Avg.	−7.46	—	—	−7.40	215.0
Total	—	32	35	—	—

^aComparison performed with the results obtained without idle time

the BKS (Time to BKS (s)). Tables 51–62 in Appendix A.3 of ESM contain detailed results for all instances considered.

We can observe that the proposed unified heuristic was capable of finding or improving the upper bound of all open instances but one, and also to obtain all known optimal solutions. More precisely, UILS was found capable of improving the result of 591 open instances and to equal the result of another 128, where 64 of them are proven optimal solutions. The total average CPU time spent by UILS ranged, on average, from approximately 3 seconds, for the 40-job instances, to roughly 561 seconds, for the 200-job instances. When compared to the gaps found by the mathematical programming-based heuristic (PR + Benders + SiPS) proposed by [Şen and Bülbül \(2015\)](#), it can be observed that the UILS clearly outperforms this method in terms of solution quality, but with higher total runtimes. However, UILS required, on average, only 3.6 seconds to find or improve the best solution of 719 (out of 720) instances.

5.7 Results for problem $R|| \sum w'_j E_j + w_j T_j$

We ran UILS for the same set of instances considered in Sect. 5.6, but now considering earliness penalties. Idle times are also permitted. We performed similar comparisons and analyses as for problem $R|| \sum w_j T_j$. Aggregated results are provided in Table 23, while detailed results are presented in Tables 63–74 (see Appendix A.4 of ESM).

From Table 23, we can observe that the proposed algorithm managed to improve the best known upper bounds of 634 instances and to equal the best results of another 84, including all known 33 optimal solutions. As expected, the CPU times are much higher than problem $R|| \sum w_j T_j$, even for $I_{LS} = n$ rather than $I_{LS} = 4 \times n$, because of the existence

of idle times, which make the local search procedure more time consuming. Moreover, UILS spent, on average, more CPU time than PR+Benders+SiPSi, but at the same time we can observe that the average gaps found by our heuristic was always smaller than those obtained by PR+Benders+SiPSi. Finally, despite the higher total runtimes, UILS required, on average, 48.8 seconds to find or improve the best solutions of 718 instances.

5.8 Summary of all results

Table 24 presents a summary of the results found for all 15,251 instances considered in this work. Here we did not consider problem $P|| \sum w'_j E_j + w_j T_j$ with idle times (see Sect. 5.5) because of the absence of results in the literature to be compared. From Table 24, it can be observed that UILS equaled the BKS in 12,434 (81.5%) of them. Moreover, out of 13,808 known optimal solutions, the proposed algorithm was found capable of obtaining 12,244 (88.7%). Finally, out of 1443 open instances, UILS managed to improve the result of 1239 (85.9%). Despite its generality, these results indicate that our unified heuristic appears to be quite robust in terms of finding high-quality solutions for several (E–)T variants.

5.9 Impact of the developed move evaluation scheme on parallel machine problems without both idle times and sequence-dependent setup times

In this section we are interested in evaluating the performance of the move evaluation scheme that was developed for parallel machine problems that do not consider idle times as well as sequence-dependent setup times. We carried out some experiments in order to compare the effect of adopting the move evaluation approach that is performed in amortized $\mathcal{O}(\max\{l, l'\})$ time, as described in Sect. 4.2.3, rather than

Şen and Bülbül (2015)

^a Single run on Intel 3.80 GHz Core i7 920 with hyperthreading and 24 GB

Table 23 Summary of results for problem $R||\sum w'_j E_j + w_j T_j$

Instance group	Sen and Bülbül (2015)				UJLS				Average			
	Best		PR+Benders+SIPSi		Best run				Gap _{LB} (%)		Time (s)	
	#Opt	Gap _{LB} (%)	#Opt	#Equal to BKS	Gap _{LB} (%)	Time ^a (s)	#Opt	#Imp	#Equal to BKS	Gap _{LB} (%)	Time to BKS (s)	Time (s)
N40_M2	22	0.16	1	1	0.96	52.9	22	13	47	0.13	2.6	22.1
N60_M2	5	0.89	0	42	0.98	109.7	5	50	10	0.43	8.7	134.4
N60_M3	4	0.82	0	20	1.58	120.2	4	37	23	0.39	18.4	109.3
N80_M2	2	0.90	0	56	0.92	134.1	2	57	3	0.36	16.9	437.2
N80_M4	0	4.54	0	58	4.57	228.3	0	60	0	2.15	7.9	317.5
N90_M3	0	2.52	0	58	2.55	153.1	0	59	1	1.27	29.8	462.9
N100_M5	0	8.83	0	60	8.83	297.7	0	60	0	6.03	17.0	519.2
N120_M3	0	4.12	0	60	4.12	165.9	0	60	0	3.21	72.0	595.4
N120_M4	0	6.98	0	60	6.98	217.1	0	60	0	5.20	45.2	593.9
N150_M5	0	13.90	0	60	13.90	279.4	0	59	0	11.73	75.1	600.0
N160_M4	0	8.60	0	60	8.60	202.5	0	60	0	7.44	131.2	600.0
N200_M5	0	11.70	0	60	11.70	257.0	0	59	0	10.16	160.5	600.0
Total	33	—	1	595	—	—	33	634	84	—	—	—
Avg.	—	5.33	—	—	5.47	184.83	—	—	—	4.04	48.8	416.0

^aSingle run on an Intel 3.80 GHz Core i7 920 with hyperthreading and 24 GB

Table 24 Summary of all experiments

Problem	IT	#Inst	#Opt	UILS						
				Best run					Average	
				#Equal to BKS	#Imp	#Opt	Gap _{LB} (%)	Gap _{UB} (%)	Gap _{LB} (%)	Gap _{UB} (%)
$1 \sum w_j T_j$		875	875	875	—	875	0.00	0.00	0.00	0.00
$1 \sum w'_j E_j + w_j T_j$		875	875	870	—	870	< 0.01	< 0.01	< 0.01	< 0.01
$1 \sum w'_j E_j + w_j T_j$	✓	5276	5276	5274	—	5274	< 0.01	< 0.01	0.07	0.07
$1 r_j \sum w'_j E_j + w_j T_j$	✓	2400	2400	1149	—	1149	0.12	0.12	0.31	0.31
$1 r_j \sum w_j C_j$	✓	1900	1900	1599	—	1599	0.01	0.01	0.02	0.02
$P \sum T_j$		2250	2250	2250	—	2250	0.00	0.00	< 0.01	< 0.01
$P \sum w_j T_j$		150	135	137	7	130	—	< 0.01	—	< 0.01
$P \sum w'_j E_j + w_j T_j$		85	—	68	7	—	—	-0.39	—	- 0.34
$R \sum w_j T_j$		720	64	128	591	64	2.56	- 1.08	2.96	- 0.88
$R \sum w'_j E_j + w_j T_j$	✓	720	33	84	634	33	4.04	- 1.11	4.27	- 0.92
Total	—	15,251	13,808	12,434	1239	12,244	—	—	—	—

Table 25 CPU time spent for problem $P|| \sum w_j T_j$ considering a single iteration of UILS_{Trad} and UILS_{Fast}

Instance group	UILS _{Trad} $t_{Avg.}$ (s)	UILS _{Fast} $t_{Avg.}$ (s)	Speedup
wt40-2m	0.48	0.39	1.23
wt40-4m	0.48	0.48	1.00
wt40-10m	0.17	0.35	0.48
wt50-2m	0.99	0.68	1.45
wt50-4m	1.03	0.84	1.23
wt50-10m	0.47	0.78	0.60
wt100-2m	19.71	7.78	2.53
wt100-4m	18.73	9.20	2.04
wt100-10m	9.20	7.62	1.21
wt200-2m	453.69	113.83	3.98
wt200-4m	450.08	142.68	3.15
wt200-10m	227.54	116.49	1.95

Table 26 CPU time spent for problem $P|| \sum w'_j E_j + w_j T_j$ considering a single iteration of UILS_{Trad} and UILS_{Fast}

Instance group	UILS _{Trad} $t_{Avg.}$ (s)	UILS _{Fast} $t_{Avg.}$ (s)	Speedup
wet40-2m	0.57	0.48	1.19
wet40-4m	0.56	0.57	0.98
wet40-10m	0.18	0.38	0.47
wet50-2m	1.72	1.22	1.41
wet50-4m	1.71	1.41	1.21
wet50-10m	0.45	0.76	0.59
wet100-2m	35.37	15.55	2.27
wet100-4m	36.02	18.24	1.97
wet100-10m	13.75	12.27	1.12
wet200-2m	1047.07	260.40	4.02
wet200-4m	842.24	267.81	3.14
wet200-10m	363.45	188.43	1.93

in $\mathcal{O}(n)$ time, as explained in Sect. 4.2.2. We consider the instances involving up to 200 jobs of problems $P||\sum w_j T_j$ and $P||\sum w'_j E_j + w_j T_j$. We executed a single iteration ($I_R = 1$) of UILS 5 times for each instance and Tables 25 and 26 report, for each group of instances, the average CPU time spent by the algorithm when using the traditional way of evaluating a move (UILS_{Trad}) and when using the enhanced way (UILS_{Fast}). In addition, we provide the speedup achieved by UILS_{Fast}, which is given by the ratio between the average CPU time spent by UILS_{Trad} and the one spent by UILS_{Fast}. Values in bold indicate that UILS_{Fast} is faster than UILS_{Trad}.

The results presented in Tables 25 and 26 demonstrate the benefits of using the enhanced move evaluation scheme, especially for large size instances containing a relative small number of machines, thus showing that the overhead of preprocessing the required data structures makes it worth to perform the move evaluation in amortized $\mathcal{O}(\max\{l, l'\})$, rather than in $\mathcal{O}(n)$. In some cases, UILS_{Fast} is around 4 times faster than UILS_{Trad}. However, the gains are less visible when the instances have a relative large number of machines. In fact, UILS_{Fast} even performs slower than UILS_{Trad} when the average number of jobs per machine is very small, meaning that the overhead of preprocessing the data structures necessary for evaluating a move in amortized $\mathcal{O}(\max\{l, l'\})$ steps for such instances does not always pay off.

6 Concluding remarks

This paper proposed a unified Iterated Local Search-based heuristic algorithm, denoted as UILS, for a large class of earliness–tardiness scheduling problems. There is a huge number of scheduling problems in the literature and they are usually solved by specific resolution methods. Because each problem has its own particularities, the development of a more general approach becomes a highly challenging task. This also partially explains the lack of unified algorithms for scheduling problems, especially when it comes to heuristics, which often have to be considerably modified to efficiently cope with different characteristics such as the inclusion of sequence-dependent setup and/or idle times. Therefore, we strongly believe that the introduction of an efficient unified heuristic approach capable of solving a number of scheduling problems can be seen as a relevant and important contribution.

Due to the large number of works related to the variants considered in the present paper, performing a complete literature review can be very tough. Hence, another contribution of this work was to present an annotated bibliography containing more than 130 related works published in the last 25 years.

In addition to dealing with a significant variety of problems, the proposed unified algorithm also takes into account

the particularities of each problem by efficiently performing the move evaluation during the local search. UILS automatically detects the most efficient method to be used according to some characteristics of the problem such as the existence of sequence-dependent setup times, release dates, idle times and so on. The efficient move evaluation schemes used in this work were mostly based on those presented in Ibaraki et al. (2005, 2008), Ergun and Orlin (2006) and Liao et al. (2012). While we directly applied the method of the first two works for problems with idle times, we had to generalize the ideas of the latter two in order to deal with parallel machines problems without idle times. This generalization was also a crucial contribution of the paper, since it improved the performance of UILS in terms of CPU time in some problems, more especially, those without sequence-dependent setup times.

UILS was tested in more than 15,000 instances from the literature and the results obtained were compared with the best ones available. The computational experiments revealed that the unified method was capable of finding high-quality solutions in competitive CPU times, except for single machine problems that include idle time, where the complex exact algorithms of Tanaka et al. (2009), Tanaka and Fujikuma (2012) spent, on average, less CPU time to find optimal solutions for instances with up to 300 jobs.

Despite the remarkable results found by the unified algorithm for the variants with available test-problems and results, the lack of existing benchmark instances prevented UILS to have its robustness better evaluated. Future avenues of research include the generation of new set of instances for the problems where it was not possible to test the proposed algorithm, as well as the development of a simple and general exact method capable of tackling the variants considered in this work, so as to provide lower bounds for the instances to be created.

Acknowledgements We would like to thank Dr. Halil Şen, Dr. Kerem Bülbül, the Associate Editor and the anonymous Reviewers for the valuable comments and suggestions. This work was partially supported by the Brazilian research agency CNPq, Grants 131192/2013-2, 234814/2014-4 and 305223/2015-1.

References

- Akyol, D., & Bayhan, G. (2008). Multi-machine earliness and tardiness scheduling problem: An interconnected neural network approach. *The International Journal of Advanced Manufacturing Technology*, 37(5–6), 576–588.
- Amorim, R. (2013). Um estudo sobre formulações matemáticas e estratégias algorítmicas para problemas de escalonamento em máquinas paralelas com penalidades de antecipação e atraso. Master's thesis, Programa de Pós-Graduação em Informática, Instituto de Computação, Universidade Federal do Amazonas, Manaus, AM, Brazil (in Portuguese).
- Amorim, R., Dias, B., de Freitas, R., & Uchoa, E. (2013). A hybrid genetic algorithm with local search approach for e/t scheduling

- problems on identical parallel machines. In: Proceedings of the 15th annual conference companion on genetic and evolutionary computation, ACM, New York, NY, USA, GECCO '13 Companion, pp. 63–64.
- Anghinolfi, D., & Paolucci, M. (2007). Parallel machine total tardiness scheduling with a new hybrid metaheuristic approach. *Computers & Operations Research*, 34(11), 3471–3490.
- Anghinolfi, D., & Paolucci, M. (2008). A new ant colony optimization approach for the single machine total weighted tardiness scheduling problem. *International Journal of Operations Research*, 5, 44–60.
- Anghinolfi, D., & Paolucci, M. (2009). A new discrete particle swarm optimization approach for the single-machine total weighted tardiness scheduling problem with sequence-dependent setup times. *European Journal of Operational Research*, 193(1), 73–85.
- Armentano, V. A., & de França Filho, M. F. (2007). Minimizing total tardiness in parallel machine scheduling with setup times: An adaptive memory-based GRASP approach. *European Journal of Operational Research*, 183(1), 100–114.
- Arroyo, J. E. C., Nunes, G. V. P., & Kamke, E. H. (2009). Iterative local search heuristic for the single machine scheduling problem with sequence dependent setup times and due dates. In G. Yu, M. Köppen, S. M. Chen, & X. Niu (Eds.), *HIS (1)* (pp. 505–510). Washington: IEEE Computer Society.
- Avella, P., Boccia, M., & D'Auria, B. (2005). Near-optimal solutions of large-scale single-machine scheduling problems. *INFORMS Journal on Computing*, 17(2), 183–191.
- Azizoglu, M., & Kirca, O. (1998). Tardiness minimization on parallel machines. *International Journal of Production Economics*, 55(2), 163–168.
- Azizoglu, M., & Kirca, O. (1999). On the minimization of total weighted flow time with identical and uniform parallel machines. *European Journal of Operational Research*, 113(1), 91–100.
- Baker, K. R., & Scudder, G. D. (1990). Sequencing with earliness and tardiness penalties: A review. *Operations Research*, 38(1), 22–36.
- Balakrishnan, N., Kanet, J. J., & Sridharan, V. (1999). Early/tardy scheduling with sequence dependent setups on uniform parallel machines. *Computers & Operations Research*, 26(2), 127–141.
- Bank, J., & Werner, F. (2001). Heuristic algorithms for unrelated parallel machine scheduling with a common due date, release dates, and linear earliness and tardiness penalties. *Mathematical and Computer Modelling*, 33(4–5), 363–383.
- Baptiste, P., Jouglet, A., & Savourey, D. (2008). Lower bounds for parallel machine scheduling problems. *International Journal of Operational Research*, 3(6), 643–664.
- Belouadah, H., & Potts, C. (1994). Scheduling identical parallel machines to minimize total weighted completion time. *Discrete Applied Mathematics*, 48(3), 201–218.
- Belouadah, H., Posner, M., & Potts, C. (1992). Scheduling with release dates on a single machine to minimize total weighted completion time. *Discrete Applied Mathematics*, 36(3), 213–231.
- Bigras, L. P., Gamache, M., & Savard, G. (2008). The time-dependent traveling salesman problem and single machine scheduling problems with sequence dependent setup times. *Discrete Optimization*, 5(4), 685–699.
- Bilge, Ü., Kırac, F., Kurtulan, M., & Pekgün, P. (2004). A tabu search algorithm for parallel machine total tardiness problem. *Computers & Operations Research*, 31(3), 397–414.
- Biskup, D., & Feldmann, M. (2001). Benchmarks for scheduling on a single machine against restrictive and unrestrictive common due dates. *Computers & Operations Research*, 28(8), 787–801.
- Biskup, D., Herrmann, J., & Gupta, J. N. (2008). Scheduling identical parallel machines to minimize total tardiness. *International Journal of Production Economics*, 115(1), 134–142.
- Bozejko, W. (2010). Parallel path relinking method for the single machine total weighted tardiness problem with sequence-dependent setups. *Journal of Intelligent Manufacturing*, 21(6), 777–785.
- Bülbül, K., & Şen, H. (2017). An exact extended formulation for the unrelated parallel machine total weighted completion time problem. *Journal of Scheduling*, 20(4), 373–389.
- Bülbül, K., Kaminsky, P., & Yano, C. (2007). Preemption in single machine earliness/tardiness scheduling. *Journal of Scheduling*, 10, 271–292.
- Chekuri, C., & Khanna, S. (2004). Approximation algorithms for minimizing average weighted completion time. In: J. Y.-T. Leung (Ed.), *Handbook of scheduling: Algorithms, models, and performance analysis* (pp. 11.1–11.30). Boca Raton, FL: CRC Press, Inc.
- Chen, Z. L., & Powell, W. B. (1999). Solving parallel machine scheduling problems by column generation. *INFORMS Journal on Computing*, 11(1), 78–94.
- Chu, C. (1992a). A branch-and-bound algorithm to minimize total flow time with unequal release dates. *Naval Research Logistics*, 39(6), 859–875.
- Chu, C. (1992b). A branch-and-bound algorithm to minimize total tardiness with different release dates. *Naval Research Logistics (NRL)*, 39(2), 265–283.
- Cicirello, V. A. (2006). Non-wrapping order crossover: An order preserving crossover operator that respects absolute position. In: *Proceedings of the genetic and evolutionary computation conference* (pp. 1125–1131), ACM Press.
- Cicirello, V. A., & Smith, S. F. (2005). Enhancing stochastic search performance by value-biased randomization of heuristics. *Journal of Heuristics*, 11, 5–34.
- Congram, R. K., Potts, C. N., & van de Velde, S. L. (2002). An iterated dynasearch algorithm for the single-machine total weighted tardiness scheduling problem. *INFORMS Journal on Computing*, 14(1), 52–67.
- Cordeau, J. F., & Maischberger, M. (2012). A parallel iterated tabu search heuristic for vehicle routing problems. *Computers & Operations Research*, 39(9), 2033–2050.
- Cordeau, J. F., Laporte, G., & Mercier, A. (2001). A unified tabu search heuristic for vehicle routing problems with time windows. *Journal of the Operational Research Society*, 52, 928–936.
- Della Croce, F. (1995). Generalized pairwise interchanges and machine scheduling. *European Journal of Operational Research*, 83(2), 310–319. **(EURO Summer Institute Combinatorial Optimization).**
- Della Croce, F., Garaix, T., & Grosso, A. (2012). Iterated local search and very large neighborhoods for the parallel-machines total tardiness problem. *Computers & Operations Research*, 39(6), 1213–1217. **(Special Issue on Scheduling in Manufacturing Systems).**
- Deng, G., & Gu, X. (2014). An iterated greedy algorithm for the single-machine total weighted tardiness problem with sequence-dependent setup times. *International Journal of Systems Science*, 45(3), 351–362.
- Derigs, U., & Vogel, U. (2014). Experience with a framework for developing heuristics for solving rich vehicle routing problems. *Journal of Heuristics*, 20(1), 75–106.
- Dyer, M. E., & Wolsey, L. A. (1990). Formulating the single machine sequencing problem with release dates as a mixed integer program. *Discrete Applied Mathematics*, 26(2–3), 255–270.
- Eom, D. H., Shin, H. J., Kwun, I. H., Shim, J. K., & Kim, S. S. (2002). Scheduling jobs on parallel machines with sequence-dependent family set-up times. *The International Journal of Advanced Manufacturing Technology*, 19(12), 926–932.
- Epstein, L., Levin, A., & van Stee, R. (2013). A unified approach to truthful scheduling on related machines. In: S. Khanna (Ed.), *Proceedings of the twenty-fourth annual ACM-SIAM symposium on discrete algorithms, SODA 2013*, New Orleans, Louisiana, USA, January 6–8, 2013, SIAM, pp. 1243–1252.

- Ergun, Ö., & Orlin, J. B. (2006). Fast neighborhood search for the single machine total weighted tardiness problem. *Operations Research Letters*, 34(1), 41–45.
- Esteve, B., Aubijoux, C., Chartier, A., & T'kindt, V. (2006). A recovering beam search algorithm for the single machine just-in-time scheduling problem. *European Journal of Operational Research*, 172(3), 798–813.
- Feldmann, M., & Biskup, D. (2003). Single-machine scheduling for minimizing earliness and tardiness penalties by meta-heuristic approaches. *Computers & Industrial Engineering*, 44(2), 307–323.
- Feng, G., & Lau, H. (2008). Efficient algorithms for machine scheduling problems with earliness and tardiness penalties. *Annals of Operations Research*, 159(1), 83–95.
- Feo, T. A., & Resende, M. G. C. (1995). Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6(2), 109–133.
- França, P. M., Mendes, A., & Moscato, P. (2001). A memetic algorithm for the total tardiness single machine scheduling problem. *European Journal of Operational Research*, 132(1), 224–242.
- Gagné, C., Price, W. L., & Gravel, M. (2002). Comparing an aco algorithm with other heuristics for the single machine scheduling problem with sequence-dependent setup times. *The Journal of the Operational Research Society*, 53(8), 895–906.
- Gagné, C., Gravel, M., & Price, W. L. (2005). Using metaheuristic compromise programming for the solution of multiple-objective scheduling problems. *Journal of the Operational Research Society*, 56(6), 687–698.
- Garey, M. R., Tarjan, R. E., & Wilfong, G. T. (1988). One-processor scheduling with symmetric earliness and tardiness penalties. *Mathematics of Operations Research*, 13(2), 330–348.
- Geiger, M. J. (2010). On heuristic search for the single machine total weighted tardiness problem—Some theoretical insights and their empirical verification. *European Journal of Operational Research*, 207(3), 1235–1243.
- Graham, R., Lawler, E., Lenstra, J., & Kan, A. (1979). Optimization and approximation in deterministic sequencing and scheduling: A survey. In: P. L. Hammer, E. J. Johnson, & B. Korte (Eds.), *Discrete optimization II. Proceedings of the advanced research institute on discrete optimization and systems applications of the systems science panel of NATO and of the discrete optimization symposium co-sponsored by IBM Canada and SIAM Banff, Aha. and Vancouver. Annals of discrete mathematics* (Vol. 5, pp. 287–326). Amsterdam: Elsevier.
- Grosso, A., Croce, F. D., & Tadei, R. (2004). An enhanced dynasearch neighborhood for the single-machine total weighted tardiness scheduling problem. *Operations Research Letters*, 32(1), 68–72.
- Guinet, A. (1995). Scheduling independent jobs on uniform parallel machines to minimize tardiness criteria. *Journal of Intelligent Manufacturing*, 6(2), 95–103.
- Guo, Q., & Tang, L. (2015). An improved scatter search algorithm for the single machine total weighted tardiness scheduling problem with sequence-dependent setup times. *Applied Soft Computing*, 29(C), 184–195.
- Guo, Y., Lim, A., Rodrigues, B., & Yu, S. (2004). Minimizing total flow time in single machine environment with release time: An experimental analysis. *Computers & Industrial Engineering*, 47(2–3), 123–140.
- Gupta, S. R., & Smith, J. S. (2006). Algorithms for single machine total tardiness scheduling with sequence dependent setups. *European Journal of Operational Research*, 175(2), 722–739.
- Hendel, Y., & Sourd, F. (2006). Efficient neighborhood search for the one-machine earliness–tardiness scheduling problem. *European Journal of Operational Research*, 173(1), 108–119.
- Hendel, Y., & Sourd, F. (2007). An improved earliness–tardiness timing algorithm. *Computers & Operations Research*, 34(10), 2931–2938.
- Ibaraki, T., Imahori, S., Kubo, M., Masuda, T., Uno, T., & Yagiura, M. (2005). Effective local search algorithms for routing and scheduling problems with general time-window constraints. *Transportation Science*, 39(2), 206–232.
- Ibaraki, T., Imahori, S., Nonobe, K., Sobue, K., Uno, T., & Yagiura, M. (2008). An iterated local search algorithm for the vehicle routing problem with convex time penalty functions. *Discrete Applied Mathematics*, 156(11), 2050–2069.
- Jouglet, A., & Savourey, D. (2011). Dominance rules for the parallel machine total weighted tardiness scheduling problem with release dates. *Computers & Operations Research*, 38(9), 1259–1266.
- Józefowska, J. (2007). *Just-in-time scheduling: Models and algorithms for computer and manufacturing systems.*, International series in operations research & management science Berlin: Springer.
- Kedad-Sidhoum, S., & Sourd, F. (2010). Fast neighborhood search for the single machine earliness–tardiness scheduling problem. *Computers & Operations Research*, 37(8), 1464–1471. (Operations Research and Data Mining in Biological Systems).
- Kedad-Sidhoum, S., Solis, Y. R., & Sourd, F. (2008). Lower bounds for the earliness–tardiness scheduling problem on parallel machines with distinct due dates. *European Journal of Operational Research*, 189(3), 1305–1316.
- Kellerer, H., Tautenhahn, T., & Woeginger, G. J. (1996). Approximability and nonapproximability results for minimizing total flow time on a single machine. In: Proceedings of the twenty-eighth annual ACM symposium on theory of computing, ACM, New York, NY, USA, STOC '96, pp. 418–426.
- Kim, S. I., Choi, H. S., & Lee, D. H. (2006). Tabu search heuristics for parallel machine scheduling with sequence-dependent setup and ready times. In M. Gavrilova, O. Gervasi, V. Kumar, C. Tan, D. Taniar, A. Laganá, Y. Mun, & H. Choo (Eds.), *Computational science and its applications—ICCSA 2006. Lecture notes in computer science* (Vol. 3982, pp. 728–737). Berlin: Springer.
- Kirlik, G., & Oğuz, C. (2012). A variable neighborhood search for minimizing total weighted tardiness with sequence dependent setup times on a single machine. *Computers & Operations Research*, 39(7), 1506–1520.
- Koulamas, C. (1997). Decomposition and hybrid simulated annealing heuristics for the parallel-machine total tardiness problem. *Naval Research Logistics (NRL)*, 44(1), 109–125.
- Kumar, V. S. A., Marathe, M. V., Parthasarathy, S., & Srinivasan, A. (2009). A unified approach to scheduling on unrelated parallel machines. *Journal of the ACM*, 56(5), 28:1–28:31.
- Lauff, V., & Werner, F. (2004). Scheduling with common due date, earliness and tardiness penalties for multimachine problems: A survey. *Mathematical and Computer Modelling*, 40(5–6), 637–655.
- Lawler, E. L. (1977). A “pseudopolynomial” algorithm for sequencing jobs to minimize total tardiness. In P. L. Hammer, B. K. EL Johnson, & G. Nemhauser (Eds.), *Studies in integer programming (Annals of discrete mathematics)* (Vol. 1, pp. 331–342). Amsterdam: Elsevier.
- Lee, J. H., Yu, J. M., & Lee, D. H. (2013). A tabu search algorithm for unrelated parallel machine scheduling with sequence- and machine-dependent setups: Minimizing total tardiness. *The International Journal of Advanced Manufacturing Technology*, 69(9–12), 2081–2089.
- Lee, Y. H., & Pinedo, M. (1997). Scheduling jobs on parallel machines with sequence-dependent setup times. *European Journal of Operational Research*, 100(3), 464–474.
- Lee, Y. H., Bhaskaran, K., & Pinedo, M. (1997). A heuristic to minimize the total weighted tardiness with sequence-dependent setups. *IIE Transactions*, 29(1), 45–52.
- Lenstra, J., Kan, A. R., & Brucker, P. (1977). Complexity of machine scheduling problems. In P. L. Hammer, B. K. EL Johnson, & G.

- Nemhauser (Eds.), *Studies in integer programming (Annals of discrete mathematics)* (Vol. 1, pp. 343–362). Amsterdam: Elsevier.
- Leonardi, S., & Raz, D. (2007). Approximating total flow time on parallel machines. *Journal of Computer and System Sciences*, 73(6), 875–891.
- Leyvand, Y., Shabtay, D., & Steiner, G. (2010). A unified approach for scheduling with convex resource consumption functions using positional penalties. *European Journal of Operational Research*, 206(2), 301–312.
- Li, G. (1997). Single machine earliness and tardiness scheduling. *European Journal of Operational Research*, 96(3), 546–558.
- Li, K., & Yang, S. (2009). Non-identical parallel-machine scheduling research with minimizing total weighted completion times: Models, relaxations and algorithms. *Applied Mathematical Modelling*, 33(4), 2145–2158.
- Li, K., Leung, J. T., & Cheng, B. Y. (2014). An agent-based intelligent algorithm for uniform machine scheduling to minimize total completion time. *Applied Soft Computing*, 25, 277–284.
- Liao, C. J., & Cheng, C. C. (2007). A variable neighborhood search for minimizing single machine weighted earliness and tardiness with common due date. *Computers & Industrial Engineering*, 52(4), 404–413.
- Liao, C. J., & Juan, H. C. (2007). An ant colony optimization for single-machine tardiness scheduling with sequence-dependent setups. *Computers & Operations Research*, 34(7), 1899–1909.
- Liao, C. J., Tsou, H. H., & Huang, K. L. (2012). Neighborhood search procedures for single machine tardiness scheduling with sequence-dependent setups. *Theoretical Computer Science*, 434, 45–52.
- Liaw, C. F., Lin, Y. K., Cheng, C. Y., & Chen, M. (2003). Scheduling unrelated parallel machines to minimize total weighted tardiness. *Computers & Operations Research*, 30(12), 1777–1789.
- Lin, S. W., & Ying, K. C. (2007). Solving single-machine total weighted tardiness problems with sequence-dependent setup times by metaheuristics. *The International Journal of Advanced Manufacturing Technology*, 34(11–12), 1183–1190.
- Lin, Y. K. (2013). Fast LP models and algorithms for identical jobs on uniform parallel machines. *Applied Mathematical Modelling*, 37(5), 3436–3448.
- Lin, Y. K., & Hsieh, F. Y. (2014). Unrelated parallel machine scheduling with setup times and ready times. *International Journal of Production Research*, 52(4), 1200–1214.
- Logendran, R., McDonnell, B., & Smucker, B. (2007). Scheduling unrelated parallel machines with sequence-dependent setups. *Computers & Operations Research*, 34(11), 3420–3438.
- Lourenço, H. R., Martin, O. C., & Stützle, T. (2002). Iterated Local Search. In: F. W. Glover, & G. A. Kochenberger (Eds.), *Handbook of metaheuristics* (pp. 321–353). Norwell, MA: Kluwer Academic Publishers.
- Mandahawi, N., Al-Shihabi, S., & Altarazi, S. (2011). A max-min ant system to minimize total tardiness on a single machine with sequence dependent setup times implementing a limited budget local search. *International Journal of Research & Reviews in Applied Sciences*, 6, 30–40.
- Martinelli, R., Poggi, M., & Subramanian, A. (2013). Improved bounds for large scale capacitated arc routing problem. *Computers & Operations Research*, 40(8), 2145–2160.
- Mason, S. J., Jin, S., & Jampani, J. (2005). A moving block heuristic for minimizing earliness and tardiness on a single machine with unrestrictive common due dates. *Journal of Manufacturing Systems*, 24(4), 328–338.
- Mason, S. J., Jin, S., & Jampani, J. (2009). A moving block heuristic to minimise earliness and tardiness costs on parallel machines. *International Journal of Production Research*, 47(19), 5377–5390.
- M’Hallah, R. (2007). Minimizing total earliness and tardiness on a single machine using a hybrid heuristic. *Computers & Operations Research*, 34(10), 3126–3142.
- M’Hallah, R., & Al-Khamis, T. (2012). Minimising total weighted earliness and tardiness on parallel machines using a hybrid heuristic. *International Journal of Production Research*, 50(10), 2639–2664.
- Mladenović, N., & Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research*, 24(11), 1097–1100.
- Nessah, R., Yalaoui, F., & Chu, C. (2008). A branch-and-bound algorithm to minimize total weighted completion time on identical parallel machines with job release dates. *Computers & Operations Research*, 35(4), 1176–1190.
- Nogueira, J. P. C. M., Arroyo, J. E. C., Villadiego, H. M. M., & Gonçalves, L. B. (2014). Hybrid GRASP heuristics to solve an unrelated parallel machine scheduling problem with earliness and tardiness penalties. *Electronic Notes in Theoretical Computer Science*, 302, 53 – 72. Proceedings of the XXXIX Latin American Computing Conference (CLEI 2013).
- Omar, M. K., & Teo, S. C. (2006). Minimizing the sum of earliness/tardiness in identical parallel machines schedule with incompatible job families: An improved MIP approach. *Applied Mathematics and Computation*, 181(2), 1008–1017.
- Pan, Y., & Shi, L. (2008). New hybrid optimization algorithms for machine scheduling problems. *IEEE Transactions on Automation Science and Engineering*, 5(2), 337–348.
- Penna, P., Subramanian, A., & Ochi, L. (2013). An iterated local search heuristic for the heterogeneous fleet vehicle routing problem. *Journal of Heuristics*, 19(2), 201–232.
- Pereira-Lopes, M. J., & Valério de Carvalho, J. M. (2007). A branch-and-price algorithm for scheduling parallel machines with sequence dependent setup times. *European Journal of Operational Research*, 176(3), 1508–1527.
- Pessoa, A., Uchoa, E., de Aragão, M., & Rodrigues, R. (2010). Exact algorithm over an arc-time-indexed formulation for parallel machine scheduling problems. *Mathematical Programming Computation*, 2(3–4), 259–290.
- Pfund, M., Fowler, J. W., Gadkari, A., & Chen, Y. (2008). Scheduling jobs on parallel machines with setup times and ready times. *Computers & Industrial Engineering*, 54(4), 764–782.
- Pisinger, D., & Røpke, S. (2007). A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34(8), 2403–2435.
- Polyakovskiy, S., & M’Hallah, R. (2014). A multi-agent system for the weighted earliness tardiness parallel machine problem. *Computers & Operations Research*, 44, 115–136.
- Potts, C. N., & Van Wassenhove, L. N. (1991). Single machine tardiness sequencing heuristics. *IIE Transactions*, 23(4), 346–354.
- Radhakrishnan, S., & Ventura, J. A. (2000). Simulated annealing for parallel machine scheduling with earliness-tardiness penalties and sequence-dependent set-up times. *International Journal of Production Research*, 38(10), 2233–2252.
- Raja, K., Selladurai, V., Saravanan, R., & Arumugam, C. (2008). Earliness–tardiness scheduling on uniform parallel machines using simulated annealing and fuzzy logic approach. *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacturing*, 222(2), 333–346.
- Raman, N., Rachamadugu, R. V., & Talbot, F. (1989). Real-time scheduling of an automated manufacturing center. *European Journal of Operational Research*, 40(2), 222–242.
- Ratli, M., Benmansour, R., Macedo, R., Hanafi, S., & Wilbaut, C. (2013). *Mathematical programming and heuristics for scheduling problems with early and tardy penalties* (pp. 183–223). Hoboken: Wiley.
- Rios-Solis, Y., & Sourd, F. (2008). Exponential neighborhood search for a parallel machine scheduling problem. *Computers & Operations Research*, 35(5), 1697–1712. (Part Special Issue: Algorithms and Computational Methods in Feasibility and Infeasibility).
- Rodrigues, R., Pessoa, A., Uchoa, E., & de Aragão, M. P. (2008). Heuristic algorithm for the parallel machine total weighted tar-

- diness scheduling problem. *Relatório de Pesquisa em Engenharia de Produção*, 8(10), 1–11.
- Ronconi, D. A. P., & Kawamura, M. A. S. (2010). The single machine earliness and tardiness scheduling problem: Lower bounds and a branch-and-bound algorithm. *Computational & Applied Mathematics*, 29, 107–124.
- Røpke, S., & Pisinger, D. (2006). A unified heuristic for a large class of vehicle routing problems with backhauls. *European Journal of Operational Research*, 171(3), 750–775.
- Rubin, P. A., & Ragatz, G. L. (1995). Scheduling in a sequence dependent setup environment with genetic search. *Computers & Operations Research*, 22(1), 85–99. (**Genetic Algorithms**).
- Schaller, J. E. (2014). Minimizing total tardiness for scheduling identical parallel machines with family setups. *Computers & Industrial Engineering*, 72, 274–281.
- Selim Akturk, M., & Ozdemir, D. (2000). An exact approach to minimizing total weighted tardiness with release dates. *IIE Transactions*, 32(11), 1091–1101.
- Şen, H., & Bülbül, K. (2015). A strong preemptive relaxation for weighted tardiness and earliness/tardiness problems on unrelated parallel machines. *INFORMS Journal on Computing*, 27(1), 135–150.
- Shim, S. O., & Kim, Y. D. (2007a). Minimizing total tardiness in an unrelated parallel-machine scheduling problem. *Journal of the Operational Research Society*, 58(3), 346–354.
- Shim, S. O., & Kim, Y. D. (2007b). Scheduling on parallel identical machines to minimize total tardiness. *European Journal of Operational Research*, 177(1), 135–146.
- Silva, M. M., Subramanian, A., Vidal, T., & Ochi, L. S. (2012). A simple and effective metaheuristic for the minimum latency problem. *European Journal of Operational Research*, 221(3), 513–520.
- Sioud, A., Gravel, M., & Gagné, C. (2012). A hybrid genetic algorithm for the single machine scheduling problem with sequence-dependent setup times. *Computers & Operations Research*, 39(10), 2415–2424.
- Sivrikaya-Şerifoğlu, F., & Ulusoy, G. (1999). Parallel machine scheduling with earliness and tardiness penalties. *Computers & Operations Research*, 26(8), 773–787.
- Sourd, F. (2005). Earliness-tardiness scheduling with setup considerations. *Computers & Operations Research*, 32(7), 1849–1865.
- Sourd, F. (2006). Dynasearch for the earliness-tardiness scheduling problem with release dates and setup constraints. *Operations Research Letters*, 34(5), 591–598.
- Sourd, F. (2009). New exact algorithms for one-machine earliness-tardiness scheduling. *INFORMS Journal on Computing*, 21(1), 167–175.
- Sourd, F., & Kedad-Sidhoum, S. (2003). The one-machine problem with earliness and tardiness penalties. *Journal of Scheduling*, 6(6), 533–549.
- Sourd, F., & Kedad-Sidhoum, S. (2008). A faster branch-and-bound algorithm for the earliness-tardiness scheduling problem. *Journal of Scheduling*, 11(1), 49–58.
- Subramanian, A. (2012). *Heuristic, exact and hybrid approaches for vehicle routing problems*. PhD thesis, Universidade Federal Fluminense, Niterói, Brazil.
- Subramanian, A., & Battarra, M. (2013). An iterated local search algorithm for the travelling salesman problem with pickups and deliveries. *Journal of the Operational Research Society*, 64(3), 402–409.
- Subramanian, A., & Farias, K. (2017). Efficient local search limitation strategy for single machine total weighted tardiness scheduling with sequence-dependent setup times. *Computers & Operations Research*, 79, 190–206.
- Subramanian, A., Drummond, L., Bentes, C., Ochi, L., & Farias, R. (2010). A parallel heuristic for the vehicle routing problem with simultaneous pickup and delivery. *Computers & Operations Research*, 37(11), 1899–1911. (**Metaheuristics for Logistics and Vehicle Routing**).
- Subramanian, A., Uchoa, E., & Ochi, L. S. (2013). A hybrid algorithm for a class of vehicle routing problems. *Computers & Operations Research*, 40(10), 2519–2531.
- Subramanian, A., Battarra, M., & Potts, C. N. (2014). An iterated local search heuristic for the single machine total weighted tardiness scheduling problem with sequence-dependent setup times. *International Journal of Production Research*, 52(9), 2729–2742.
- Sun, H., & Wang, G. (2003). Parallel machine earliness and tardiness scheduling with proportional weights. *Computers & Operations Research*, 30(5), 801–808.
- Tan, K., & Narasimhan, R. (1997). Minimizing tardiness on a single processor with sequence-dependent setup times: A simulated annealing approach. *Omega*, 25(6), 619–634.
- Tanaka, S. (2012). An exact algorithm for the single-machine earliness-tardiness scheduling problem. In R. Z. Ríos-Mercado & Y. A. Ríos-Solís (Eds.), *Just-in-time systems, Springer optimization and its applications* (pp. 21–40). New York: Springer.
- Tanaka, S., & Araki, M. (2008). A branch-and-bound algorithm with lagrangian relaxation to minimize total tardiness on identical parallel machines. *International Journal of Production Economics*, 113(1), 446–458.
- Tanaka, S., & Araki, M. (2013a). An exact algorithm for the single-machine total weighted tardiness problem with sequence-dependent setup times. *Computers & Operations Research*, 40(1), 344–352.
- Tanaka, S., & Araki, M. (2013b). An exact algorithm for the single-machine total weighted tardiness problem with sequence-dependent setup times. *Computers & Operations Research*, 40(1), 344–352.
- Tanaka, S., & Fujikuma, S. (2008). An efficient exact algorithm for general single-machine scheduling with machine idle time. In: *IEEE international conference on automation science and engineering, 2008. CASE 2008*, pp 371–376.
- Tanaka, S., & Fujikuma, S. (2012). A dynamic-programming-based exact algorithm for general single-machine scheduling with machine idle time. *Journal of Scheduling*, 15(3), 347–361.
- Tanaka, S., Fujikuma, S., & Araki, M. (2009). An exact algorithm for single-machine scheduling without machine idle time. *Journal of Scheduling*, 12(6), 575–593.
- Tasgetiren, M. F., Pan, Q. K., & Liang, Y. C. (2009). A discrete differential evolution algorithm for the single machine total weighted tardiness problem with sequence dependent setup times. *Computers & Operations Research*, 36(6), 1900–1915.
- Tsai, T. (2007). A genetic algorithm for solving the single machine earliness/tardiness problem with distinct due dates and ready times. *The International Journal of Advanced Manufacturing Technology*, 31(9), 994–1000.
- Valente, J. M., & Alves, R. A. (2008). Beam search algorithms for the single machine total weighted tardiness scheduling problem with sequence-dependent setups. *Computers & Operations Research*, 35(7), 2388–2405.
- Vallada, E., & Ruiz, R. (2012). Scheduling unrelated parallel machines with sequence dependent setup times and weighted earliness-tardiness minimization. In R. Z. Ríos-Mercado & Y. A. Ríos-Solís (Eds.), *Just-in-time systems, Springer optimization and its applications* (Vol. 60, pp. 67–90). New York: Springer.
- Van den Akker, J., Hurkens, C., & Savelsbergh, M. (2000). Time-indexed formulations for machine scheduling problems: Column generation. *INFORMS Journal on Computing*, 12(2), 111–124.
- Vidal, T., Crainic, T. G., Gendreau, M., & Prins, C. (2013). A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows. *Computers & Operations Research*, 40(1), 475–489.

- Vidal, T., Crainic, T. G., Gendreau, M., & Prins, C. (2014). A unified solution framework for multi-attribute vehicle routing problems. *European Journal of Operational Research*, 234(3), 658–673.
- Vidal, T., Battarra, M., Subramanian, A., & Erdoğan, G. (2015a). Hybrid metaheuristics for the clustered vehicle routing problem. *Computers & Operations Research*, 58, 87–99.
- Vidal, T., Crainic, T. G., Gendreau, M., & Prins, C. (2015b). Timing problems and algorithms: Time decisions for sequences of activities. *Networks*, 65(2), 102–128.
- Wan, G., & Yen, B. P. C. (2002). Tabu search for single machine scheduling with distinct due windows and weighted earliness/tardiness penalties. *European Journal of Operational Research*, 142(2), 271–281.
- Wan, L., & Yuan, J. (2013). Single-machine scheduling to minimize the total earliness and tardiness is strongly np-hard. *Operations Research Letters*, 41(4), 363–365.
- Webster, S. (1992). New bounds for the identical parallel processor weighted flow time problem. *Management Science*, 38(1), 124–136.
- Webster, S. (1995). Weighted flow time bounds for scheduling identical processors. *European Journal of Operational Research*, 80(1), 103–111.
- Webster, S. T. (1993). A priority rule for minimizing weighted flow time in a class of parallel machine scheduling problems. *European Journal of Operational Research*, 70(3), 327–334.
- Weng, M. X., Lu, J., & Ren, H. (2001). Unrelated parallel machine scheduling with setup consideration and a total weighted completion time objective. *International Journal of Production Economics*, 70(3), 215–226.
- Xi, Y., Jang, J., Friedman, D., & Hou, W. (2015). A tardiness-concerned constructive method for the identical parallel machine scheduling. *The International Journal of Advanced Manufacturing Technology*, 79(5–8), 851–862.
- Xu, H., Lü, Z., & Cheng, T. (2013). Iterated local search for single-machine scheduling with sequence-dependent setup times to minimize total weighted tardiness. *Journal of Scheduling*, 17, 271–287.
- Xu, H., Lü, Z., Yin, A., Shen, L., & Buscher, U. (2014). A study of hybrid evolutionary algorithms for single machine scheduling problem with sequence-dependent setup times. *Computers & Operations Research*, 50, 47–60.
- Yalaoui, F., & Chu, C. (2002). Parallel machine scheduling to minimize total tardiness. *International Journal of Production Economics*, 76(3), 265–279.
- Yalaoui, F., & Chu, C. (2006). New exact method to solve the $P_m|r_j|\sum C_j$ schedule problem. *International Journal of Production Economics*, 100(1), 168–179.
- Yano, C. A., & Kim, Y. D. (1991). Algorithms for a class of single-machine weighted tardiness and earliness problems. *European Journal of Operational Research*, 52(2), 167–178.
- Ying, K. C., Lin, S. W., & Huang, C. Y. (2009). Sequencing single-machine tardiness problems with sequence dependent setup times using an iterated greedy heuristic. *Expert Systems with Applications*, 36(3, Part 2), 7087–7092.
- Yoon, S. H., & Lee, I. S. (2011). New constructive heuristics for the total weighted tardiness problem. *Journal of the Operational Research Society*, 62(1), 232–237.
- Yousefi, M., & Yusuff, R. M. (2013). Minimising earliness and tardiness penalties in single machine scheduling against common due date using imperialist competitive algorithm. *International Journal of Production Research*, 51(16), 4797–4804.
- Zhou, H., Li, Z., & Wu, X. (2007). Scheduling unrelated parallel machine to minimize total weighted tardiness using ant colony optimization. In: *2007 IEEE international conference on automation and logistics*, pp 132–136.
- Zhu, Z., & Heady, R. B. (2000). Minimizing the sum of earliness/tardiness in multi-machine scheduling: A mixed integer programming approach. *Computers & Industrial Engineering*, 38(2), 297–305.